

Mamba2Diff: An enhanced diffusion framework for goal-conditioned imitation learning in robotic long-horizon action modeling

Guangpeng Zhao ^a, Quan Liu ^a, Wupeng Deng ^b, Jiwei Hu ^{a,*}

^a School of Information Engineering, Wuhan University of Technology, 430070, Wuhan, China

^b Hubei Key Laboratory of Modern Manufacturing Quality Engineering, School of Mechanical Engineering, Hubei University of Technology, 430068, Wuhan, China

ARTICLE INFO

Keywords:

Goal conditioned imitation learning
Robotic manipulation
Diffusion model
Long horizon tasks
State space models

ABSTRACT

Executing long-horizon, high-precision, goal-oriented tasks is essential for enabling robots to perform complex manipulation procedures in dynamic environments. In light of this, Goal-Conditioned Imitation Learning (GCIL) leverages human demonstrations to derive goal-oriented policies, demonstrating significant promise in autonomous robotic manipulation of intricate tasks. However, current GCIL approaches face challenges in capturing both long- and short-term dependencies and in generating consistent behaviors, especially within high-dimensional and dynamic environments. To address these limitations, we propose a novel diffusion-based framework, Mamba2Diff, which aims to improve policy generation by effectively capturing temporal dependencies from multi-modal inputs, thereby enhancing both action consistency and generation accuracy. The core of Mamba2Diff includes the Bidirectional Gated Mamba2 (BDGM) module, which extracts global features through bidirectional information flow, ensuring stability and consistency in long-horizon action modeling. Additionally, a Convolution-Enhanced Gated Recurrent Unit (CGRU) module is incorporated to efficiently capture short-term dependencies, balancing long- and short-term information to further improve generation precision. The effectiveness of Mamba2Diff is evaluated across three simulated environments and a customized real-world robotic platform. The results demonstrate that Mamba2Diff consistently outperforms both diffusion-based and non-diffusion-based baselines in long-horizon manipulation tasks, achieving superior performance in terms of precision, efficiency, and temporal consistency, with significant advantages observed during the later stages of task execution. These findings highlight the model's capacity to generate reliable and accurate robotic behaviors, underscoring its potential for deployment in dynamic, real-world applications.

1. Introduction

Executing long-horizon, high-precision, and goal-oriented tasks is crucial for enabling robots to perform complex manipulation activities in real-world environments [1–4]. Traditional control algorithms, while foundational in robotic manipulation, often fall short in dynamic and unpredictable settings due to their limited generalizability and the necessity for extensive parameter tuning [5,6]. These limitations hinder the effective adaptation to diverse and evolving task demands. To address these challenges, conventional imitation learning (IL) methods were developed to guide robots by using explicitly designed rewards, aiming to improve task performance in dynamic and complex environments [6,7]. However, these methods are typically task-specific and heavily reliant on high-quality expert demonstrations, which limits their scalability and adaptability in multi-task settings. Goal-Conditioned Imitation Learning (GCIL) addresses these issues by removing the need for complex rewards and incorporating explicit goal information to guide learning. This en-

ables robots to handle a broader range of tasks [8,9]. Nevertheless, as task duration increases, GCIL encounters critical challenges in balancing consistency and precision, particularly in tasks requiring fine-grained manipulation.

In the field of Goal-Conditioned Imitation Learning (GCIL), existing approaches can be broadly categorized into non-diffusion-based and diffusion-based methods. Non-diffusion-based methods, using generative models or sequence modeling architectures, often require complex architectures or multiple learning objectives, such as separate low- and high-level policies, which complicates the tuning process. In contrast, diffusion-based methods, leveraging iterative denoising processes, have been increasingly applied to model high-dimensional, dynamic action spaces and improve the generation of action sequences. Diffusion-based methods leverage iterative denoising to progressively refine action sequences, enabling the capture of complex behaviors. Despite these improvements, existing diffusion-based GCIL methods still face two key challenges in long-horizon action modeling. The core

* Corresponding author.

E-mail addresses: 276223@whut.edu.cn (G. Zhao), huijiwei@whut.edu.cn (J. Hu).

denoising architectures commonly employed, such as Unet [10], CNNs [11], and Transformers [12], often fail to capture long-term dependencies adequately, making it difficult to maintain behavioral consistency over extended action sequences. This limitation may cause goal deviations during task execution, especially in dynamic, complex scenarios where a coherent policy is essential. Additionally, these architectures typically do not adequately incorporate short-term dependencies, resulting in reduced precision in the generated actions and fine-grained operations. The inability to balance long-term consistency with short-term precision undermines the overall quality of action generation, highlighting the need for enhanced consistency and precision in diffusion-based GCIL frameworks to achieve high-quality long-horizon sequence modeling.

To tackle these challenges, We propose a novel diffusion-based generation framework, Mamba2Diff, with a core denoising network integrates the Mamba2 module [13], specifically to support long-horizon and multi-modal behavior generation in robotic manipulation. Leveraging Mamba2's efficiency in multi-modal information processing and long-sequence modeling [14,15], Mamba2Diff enables stable behavior generation in high-dimensional, dynamic environments.

Mamba2Diff introduces a bidirectional Mamba2 structure [16,17] that integrates forward and reverse information flows to capture comprehensive global contextual information, ensuring consistency and goal-oriented action sequences in long-horizon tasks. To further optimize the fusion of bidirectional information, we propose an Adaptive Allocation Gate (AAG) module to adaptively balance information flow based on task requirements. The combination of bidirectional Mamba2 and AAG constitutes the Bidirectional Gated Mamba2 (BDGM), which achieves more stable and consistent behavior generation over long horizons, enhancing the execution capabilities of robots in complex environments.

Additionally, Mamba2Diff incorporates a Convolution-Enhanced Gated Recurrent Unit (CGRU) module, which combines convolutional processing with Gated Recurrent Unit (GRU)-based temporal modeling to capture both local spatial dynamics and sequential dependencies. This integration enables the model to track real-time changes in the environment and adapt to short-term variations, refining control over actions. By balancing long- and short-term dependencies, the design ensures consistency in long action sequences while enhancing accuracy and precision, particularly in complex, long-horizon tasks. As a result, Mamba2Diff enables accurate, coherent task executions, significantly improving robotic performance in dynamic environments.

The primary contributions of this work are as follows:

1. Mamba2Diff is proposed to address the challenges of robotic manipulation in complex, dynamic environments, with a specific focus on long-horizon tasks where it achieves high-precision action modeling via a multiscale, dependency-driven diffusion architecture, and enables real-time control through the integration of a dedicated improved denoising network architecture and a tailored efficient action generation process.
2. The proposed BDGM module leverages a bidirectional Mamba2 structure in conjunction with an AAG to effectively capture long-term dependencies, substantially improving the stability and consistency of task execution in challenging and dynamic settings.
3. The CGRU module integrates convolutional processing with GRU-based temporal modeling to capture short-term dependencies, boosting the precision of actions, particularly in tasks that require meticulous manipulation and fine-grained control.
4. Experimental results validate the superiority of Mamba2Diff, demonstrating enhanced consistency and precision compared to state-of-the-art (SOTA) methods across three virtual long-horizon tasks and a custom-designed real-world environment.

2. Preliminaries

2.1. Problem formulation

The objective of GCIL [8,18,19] is to generate goal-oriented policies by learning from expert demonstrations. Given a dataset composed of observation-action-goal triples defined as $D = \bigcup_{k=1}^K \{(o_n^k, \tau_n^k, g_k)\}_{n=1}^{N_k}$, where K represents the total number of trajectories and N_k denotes the length of the k -th trajectory with $\xi_k = \{(o_n^k, \tau_n^k)\}_{n=1}^{N_k}$ represents the sequence of observation-action pairs for the k -th trajectory, and g_k is the corresponding goal state, GCIL aims to train a policy $\pi_\theta(\tau|o, g)$ that can generate the optimal action τ given the observation o and the goal g . To achieve this, the policy network learns by maximizing the following log-likelihood objective function:

$$L_{\text{GCIL}} = \mathbb{E}_{(o, \tau, g) \sim D} [\log \pi_\theta(\tau|o, g)] \quad (1)$$

Due to the extended time horizon and complexity of tasks, the policy network must be capable of long-horizon planning. This involves not only generating actions for the current observation but also considering future decisions to ensure consistent progress toward the goal. Effective long-horizon planning is essential for handling the complexities of dynamic tasks and achieving goals through multi-step decision-making.

2.2. Preliminaries on mamba

The Mamba architecture [20] extends State Space Models (SSMs) [21] by integrating a selective mechanism to enhance representational capacity and computational efficiency. SSMs model the temporal dynamics of input-output sequences with latent states. Mamba incorporates an adaptive gating function $\varphi(\cdot)$, which allows for selective information flow in the following manner:

$$h_t = A_d h_{t-1} + B_d \varphi(x_t). \quad (2)$$

where $h_t \in \mathbb{R}^N$ is the hidden state, $x_t \in \mathbb{R}^D$ is the input at time t , and A_d, B_d are learnable parameters.

Building on Mamba, Mamba2 [13] introduces a hierarchical gating mechanism to capture long-range dependencies, which improves the model's ability to handle long sequences. The hidden state is updated as follows:

$$h_t = A_d h_{t-1} + \sum_{m=1}^M B_{d,m} \varphi_m(x_{t-\lambda_m}). \quad (3)$$

where $\varphi_m(\cdot)$ is the gating function at the m -th temporal scale λ_m , and M is the number of hierarchical levels. This mechanism allows Mamba2 to learn both fine-grained and long-range temporal dependencies, depending on the configuration of temporal scales λ_m and the number of levels M . By capturing information at multiple temporal scales, Mamba2 can more effectively retain relevant context from earlier time steps, enabling efficient modeling of long-horizon dependencies.

To enhance scalability and reduce the number of parameters, Mamba2 employs low-rank approximations:

$$\begin{aligned} A &\approx U \Sigma V^\top, \\ B &\approx \tilde{U} \tilde{\Sigma} \tilde{V}^\top, \end{aligned} \quad (4)$$

where $U, V, \tilde{U}, \tilde{V} \in \mathbb{R}^{N \times r}$ and $\Sigma, \tilde{\Sigma} \in \mathbb{R}^{r \times r}$, with $r \ll N$. This optimization improves computational efficiency, making Mamba2 particularly effective for long-horizon sequence modeling tasks [22].

3. Related work

3.1. Goal-conditioned imitation learning

GCIL is a subdomain of IL that focuses on training agents, particularly robots, to achieve specific goals by conditioning the learning process on goal information. GCIL methods can generally be categorized into two main classes: non-diffusion-based methods and diffusion-based

methods. Non-diffusion-based approaches typically employ generative models and sequence modeling architectures to enable agents to generate goal-oriented actions in multimodal environments. Notable examples of these architectures include Multi-Layer Perceptrons (MLPs) [23,24], Conditional Variational Autoencoders (CVAE) [24], and Transformers [25]. These models excel at capturing latent representations of multimodal behaviors that an agent may exhibit to achieve a given goal. In contrast, diffusion-based methods in GCIL leverage the iterative denoising process of Denoising Diffusion Probabilistic Models (DDPMs) [26,27] and Score-Based Generative Models (SGMs) [10,19,28] to generate high-quality, goal-directed action sequences. This process begins with noisy action sequences, which are progressively refined toward the target goal through a series of denoising steps. By conditioning the diffusion process on both the current state and the desired goal, these methods are capable of generating precise long-horizon action sequences.

3.2. Diffusion models in robotic manipulation

Diffusion models were initially applied in image generation with DDPMs, as proposed in [29], demonstrating high-quality image generation through progressive denoising from noise. This technique was later adapted for robotic manipulation. However, its application does not involve directly generating control signals but instead supports robotic tasks by producing visual information. For example, Constrained-Context Conditional Diffusion Models (C3DM) proposed by Saxena et al. [30] illustrate how task-relevant visual information can be generated to help robots disregard environmental distractions and focus on key areas of the task. Despite its benefits, this method's limitation lies in the separation of perception and action during control, which can introduce delays and reduce control accuracy in highly complex environments.

Beyond image generation, diffusion models have also been employed to generate reward functions for optimizing robot policies. For instance, the method proposed in [31] demonstrates how relative reward functions can be derived by comparing diffusion models with low and high rewards. However, a limitation of this approach is that the generated reward functions may generalize poorly to new tasks and environments, thereby restricting their applicability across varied scenarios.

In addition to generating visual information and reward functions, diffusion models are increasingly used for direct action sequence generation in robotic manipulation. DDPM and SGMs have shown strong capabilities in generating high-dimensional action sequences by modeling multimodal distributions. [27] introduced the Diffusion Policy, which leverages DDPM to generate vision-based action policies for robots by gradually denoising action sequences to produce high-dimensional conditioned actions, enabling robots to perform complex tasks. However, the multi-step denoising process of this approach leads to lower inference efficiency, making it challenging to meet the real-time requirements of robotic manipulation.

Additionally, Reuss et al. [19] proposed Behavior Generation with Score-Based Generative Policies (BESO), illustrating how score-based models can directly generate goal-oriented action sequences. Unlike reward-function-based methods, this approach models the multimodal distribution of robotic actions, allowing robots to execute goal-oriented tasks in complex environments. However, as task complexity and time horizons increase, this method reveals certain limitations in maintaining behavioral consistency and long-term planning, particularly in long-horizon tasks.

3.3. Denoising network structure design

Diffusion models utilize denoising networks to iteratively extract meaningful features, progressively transforming random noise into high-quality outputs that align with target distributions. In robotic manipulation, the Unet architecture functions as a denoising network, generating trajectories through its encoder-decoder structure to support path planning in varied environments. The Render and Diffuse (R&D) proposed

in [32] leveraging a Unet-based denoising network to integrate robot actions and RGB observations in a unified image space, thus enhancing spatial generalization and sample efficiency in multimodal robotic tasks. Carvalho et al. [10] applied Unet in their motion planning diffusion model, successfully generating high-dimensional motion trajectories to address intricate path planning challenges. However, the local convolution operations of Unet struggle to capture global temporal dependence in high-dimensional, long-sequence tasks, reducing the inference efficiency needed for real-time decision-making.

CNNs have also been employed for precise action trajectory generation in complex environments due to their ability to capture local features. In the DiPPeR method proposed in [11], CNNs function as denoising networks within a diffusion model, significantly accelerating the inference process for path planning. Nonetheless, CNNs are limited by their computational complexity and localized feature extraction, which hampers denoising effectiveness in long-sequence tasks that require global contextual information.

Transformers, in comparison, excel in high-dimensional signal denoising tasks by establishing long-term dependencies within input sequences through attention mechanisms. Hou et al. [12] employs a causal transformer as a denoising network, leveraging attention-based temporal dependencies to effectively align multimodal inputs and generate coherent, goal-oriented action sequences for complex robotic manipulation. Xian et al. [33] introduced the ChainedDiffuser method, which integrates a global transformer action predictor with a local trajectory diffuser, enabling the understanding of complex scene semantics while generating trajectory segments that bridge these macro actions. However, the computational complexity of Transformers increases substantially with larger input sizes, posing a significant obstacle to real-time robotic control.

To address these challenges, we select Mamba2 as the denoising network for its efficiency in handling long-term dependencies and superior computational performance. Mamba2 combines the strengths of selective SSMs with deep learning, enabling dynamic adjustments in feature extraction to meet diverse task requirements while demonstrating enhanced robustness in complex environments and with multimodal inputs.

4. Methodology

As shown in Fig. 1(a), we propose Mamba2Diff, a novel method for robot motion generation in long-horizon tasks. It introduces the Multi-scale Dependencies Fusion Block (MDFB) to enhance the denoising network's ability to capture both long-term and short-term dependencies. By building on SGMs, Mamba2Diff generates smooth, high-quality motion sequences, improving both efficiency and precision, making it ideal for continuous control in complex tasks.

4.1. Score based generative models

In our framework, we employ the Score-Based Generative Model (SGM) [19,28] for high-quality action generation in robotic path planning. SGM utilizes a reverse diffusion process to remove noise over time, transforming Gaussian noise into action sequences that align with the target distribution. The update follows:

$$d\tau \approx \frac{\tau - D_\theta(\tau, o, g, \sigma_t)}{t} dt \quad (5)$$

where τ represents the robot's action at time t , o is the observation, and g is the goal. The denoising network D_θ is trained via denoising score matching, learning the score function of the target distribution [28].

To enhance performance, we incorporate preprocessing layers, skip connections, and noise-dependent weight functions:

$$D_\theta(\tau, o, g, \sigma_t) = c_{\text{skip}}(\sigma_t)\tau + c_{\text{out}}(\sigma_t)F_\theta(c_{\text{in}}(\sigma_t)\tau, c_{\text{noise}}(\sigma_t), o, g) \quad (6)$$

where $c_{\text{skip}}(\sigma_t)$, $c_{\text{out}}(\sigma_t)$, $c_{\text{in}}(\sigma_t)$, and $c_{\text{noise}}(\sigma_t)$ [34] are weight functions that vary with the noise level σ_t , modulating the output accordingly.

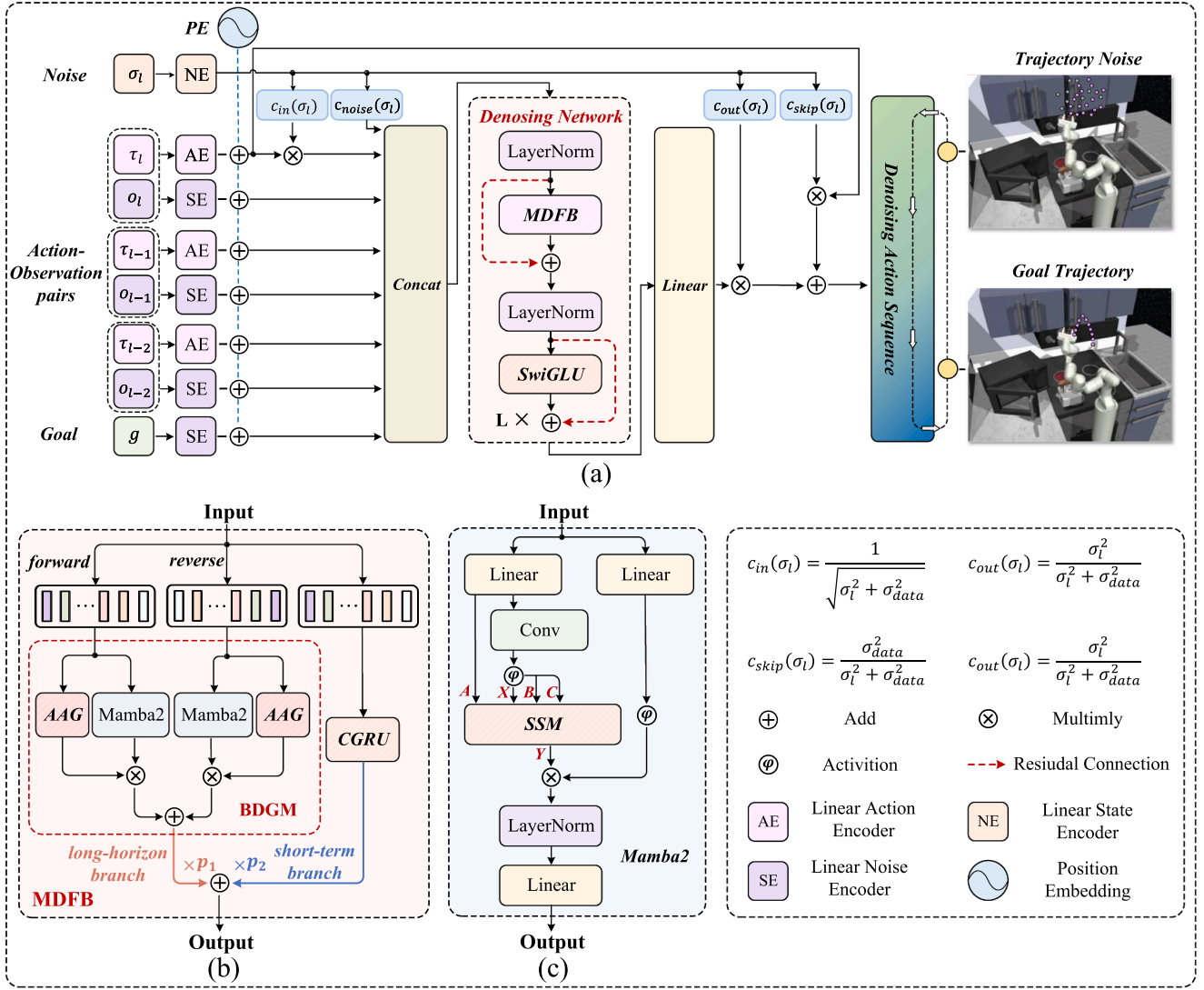


Fig. 1. The overview of our proposed Mamba2Diff action sequence generation model along with its overall architecture is provided. (a): The general process for generating action sequences using our proposed internal denoising score model employs a residual cascade structure that integrates MDFB and SwiGLU involves transforming a noisy trajectory into a goal trajectory within a diffusion process spanning l steps. This process entails receiving the current observation o , the desired goal g , and the observation-action pairs (o, τ) from the previous $l-1$ moments ($l=3$ is taken for illustration, which is proven to be highly efficient in the experiments below). The denoised action sequence is then produced by our proposed high-level score model, which incorporates preprocessing layers and a skip-connection structure. (b): The structure of the MDFB we proposed can be adapted to the modeling capabilities of complex robotic manipulation tasks. (c): Detailed overview of the Mamba2 module.

These functions are defined as:

$$\begin{aligned}
 c_{skip}(\sigma_l) &= \frac{\sigma_{data}^2}{\sigma_{data}^2 + \sigma_l^2}, \\
 c_{out}(\sigma_l) &= \frac{\sigma_l^2}{\sigma_{data}^2 + \sigma_l^2}, \\
 c_{in}(\sigma_l) &= \frac{1}{\sqrt{\sigma_{data}^2 + \sigma_l^2}}, \\
 c_{noise}(\sigma_l) &= 0.25 \log(\sigma_l)
 \end{aligned} \tag{7}$$

The model is optimized by minimizing the following loss function:

$$L(\theta) = \mathbb{E}_{\sigma_l, \tau, \eta} [\alpha(\sigma_l) \| D_\theta(\tau + \eta, o, g, \sigma_l) - \tau \|_2^2] \tag{8}$$

where $\alpha(\sigma_l)$ adjusts the weight of each noise level, and $\eta \sim \mathcal{N}(0, \sigma_l^2 I)$ represents Gaussian noise added to the action samples τ . Through this training, the model progressively refines noisy samples, enabling the

generation of action sequences that align with the target distribution [35].

4.2. Multiscale dependencies fusion block

As shown in the Fig. 1(b), The MDFB comprises two primary components: the BDGM module, designed for capturing long-term dependencies in the multimodal temporal representation, which constitutes the input to the denoising network, and the CGRU module, optimized for capturing short-term dependencies. This configuration enables the MDFB to effectively capture both global contextual information and local features, enhancing its capacity to model complex robotic manipulation tasks.

The multimodal temporal representation is denoted by the set $X = \{x_{t_l} \mid t_l \in (1, N), x_{t_l} \in \mathbb{R}^{F \times D}\}$, where each x_{t_l} represents the feature matrix at time t_l , corresponding to the discrete denoising step l . Here, l indicates the current step in the reverse diffusion process, t_l corresponds

Algorithm 1: Adaptive Allocation Gate (AAG).**Require:** Multimodal temporal representation

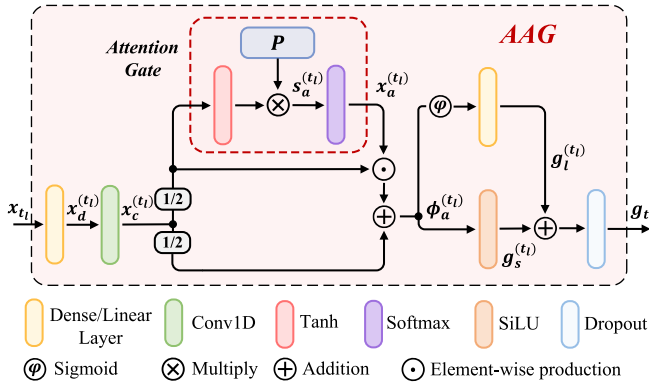
$$X = (x_1, x_2, \dots, x_N)$$

Require: Trained weight $W^{(d)}$ and bias $b^{(d)}$ of dense layer**Require:** Trained weight $W^{(e)}$ and bias $b^{(e)}$ of linear layer**Require:** Additive parameter matrix P

```

1 for  $t_l \in \{0, 1, \dots, N\}$  do
2    $x_d^{(t_l)} \leftarrow x_{t_l} W^{(d)} + b^{(d)}$ 
3    $x_c^{(t_l)} \leftarrow \text{Conv1D}(x_d^{(t_l)})$ 
4    $s_a^{(t_l)} \leftarrow P^T \tanh\left(\frac{1}{2} x_c^{(t_l)}\right)$ 
5    $x_a^{(t_l)} \leftarrow \text{softmax}(s_a^{(t_l)})$ 
6    $\phi_a^{(t_l)} \leftarrow \frac{1}{2} x_c^{(t_l)} (1 + x_a^{(t_l)})$ 
7    $g_l^{(t_l)} \leftarrow \sigma(\phi_a^{(t_l)} W^{(e)} + b^{(e)}), g_s^{(t_l)} \leftarrow \text{SiLU}(\phi_a^{(t_l)})$ 
8    $g_{t_l} \leftarrow g_l^{(t_l)} + g_s^{(t_l)}$ 
9 end
Return:  $G = (g_1, g_2, \dots, g_N)$ 

```

**Fig. 2.** The architecture of Adaptive Allocation Gate (AAG).

to the time value at the discrete step l and N denotes the total number of time points needed to generate a complete trajectory. As shown in the Fig. 1(a), this representation is assembled by concatenating the current observation o , desired goal g , time-step information, observation-action pairs (o, τ) from the preceding $l-1$ steps, and the current noise level. Each component is linearly encoded and concatenated to form the feature matrices, where F is the number of stacked features, and D represents the feature dimensionality.

Adaptive Allocation Gate: We introduce the AAG, a gating mechanism designed to optimize information flow within the network by employing dynamic weighting to enhance the performance of both the forward and backward Mamba2 branches during the information fusion process. As displayed in Fig. 2, this module first extracts local feature representations from the input feature matrix x_{t_l} via a dense layer, followed by a 1D convolutional layer. The calculation is specified as follows:

$$x_c^{(t_l)} = \text{Conv1D}(x_{t_l} W^{(d)} + b^{(d)}) \quad (9)$$

where $W^{(d)}$ is the weight matrix of the dense layer and $b^{(d)}$ is the bias term. Through this convolutional operation, the model efficiently aggregates local temporal information. To balance the contributions from parallel pathways, the output $x_c^{(t_l)}$ is scaled by a factor of $\frac{1}{2}$ before being passed to the attention gate to capture global features. The attention gate is computed as follows:

$$\begin{aligned} s_a^{(t_l)} &= P^T \tanh\left(\frac{1}{2} x_c^{(t_l)}\right), \\ x_a^{(t_l)} &= \text{softmax}(s_a^{(t_l)}) \end{aligned} \quad (10)$$

where P^T is the additive parameter matrix. To retain the local information derived from the convolution operation while obtaining global features, AAG employs a residual connection that incorporates the balanced pathway output, $\frac{1}{2} x_c^{(t_l)}$, which bypasses the attention gate, and adds it directly to the attention output, forming the enhanced representation $\phi_a^{(t_l)}$:

$$\phi_a^{(t_l)} = \frac{1}{2} x_c^{(t_l)} (1 + x_a^{(t_l)}) \quad (11)$$

Next, to regulate the selective flow of information, AAG introduces a sigmoid-based forget gate and a SiLU activation function to generate dynamic weights based on the enhanced representation $\phi_a^{(t_l)}$. Specifically, $\phi_a^{(t_l)}$ follows two parallel paths: one path undergoes sigmoid activation to generate the forget gate, followed by linear transformation, while the other path applies the SiLU activation for nonlinear mapping. The respective computations are as follows:

$$\begin{aligned} g_l^{(t_l)} &= \sigma(\phi_a^{(t_l)} W^{(e)} + b^{(e)}), \\ g_s^{(t_l)} &= \text{SiLU}(\phi_a^{(t_l)}) \end{aligned} \quad (12)$$

where $W^{(e)}$ is the weight matrix of the linear layer and $b^{(e)}$ is the bias term. The outputs of these two paths, $g_l^{(t_l)}$ and $g_s^{(t_l)}$, are then combined through additive fusion to produce the final output of the AAG:

$$g_{t_l} = g_l^{(t_l)} + g_s^{(t_l)} \quad (13)$$

Through this design, the BDGM module effectively balances the input feature matrix across both the forward and backward Mamba2 branches, resulting in a comprehensive global context representation g_{t_l} . The information flow control process of the AAG module is outlined in Algorithm 1.

Bidirectional Gated Mamba2: The BDGM module is designed primarily to model long-horizon action sequences by combining bidirectional Mamba2 paths with the AAG, thereby capturing global contextual information from the input feature matrix x_{t_l} . x_{t_l} is processed directly through the forward Mamba2 path (refer to Fig. 1(c) for the detailed structure of Mamba2), while the backward path requires reversing the matrix along the temporal dimension prior to processing.

$$\begin{aligned} m_f^{(t_l)} &= \text{Mamba2}(x_{t_l}), \\ m_b^{(t_l)} &= \text{Mamba2}(\text{Reverse}(x_{t_l})) \end{aligned} \quad (14)$$

The AAG modules then generate adaptive weights $g_f^{(t_l)}$ and $g_b^{(t_l)}$ for the forward and backward paths, respectively, facilitating the weighted integration of bidirectional information:

$$\begin{aligned} g_f^{(t_l)} &= \text{AAG}(x_{t_l}), \\ g_b^{(t_l)} &= \text{AAG}(\text{Reverse}(x_{t_l})) \end{aligned} \quad (15)$$

Finally, the global contextual representation $y_{\text{BDGM}}^{(t_l)}$ is derived by combining the outputs from both directions:

$$y_{\text{BDGM}}^{(t_l)} = g_f^{(t_l)} \cdot m_f^{(t_l)} + g_b^{(t_l)} \cdot m_b^{(t_l)} \quad (16)$$

Convolution Enhanced GRU: As demonstrated in Fig. 3, the CGRU module combines one-dimensional convolutional layers with a GRU [36,37] module to effectively capture short-term dependencies and local features in multimodal temporal representations [38]. Initially, the input feature matrix x_{t_l} is processed through a 1D convolutional layer to extract localized features, generating a localized feature representation c_{t_l} :

$$c_{t_l} = \text{Conv1D}(x_{t_l}) \quad (17)$$

The CGRU module then utilizes the local feature representation c_{t_l} to capture further short-term temporal dependencies. The CGRU's

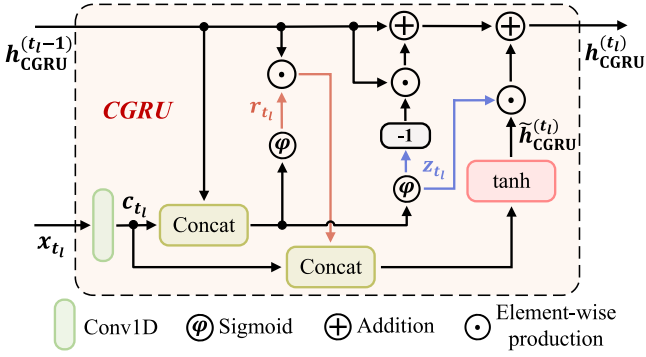


Fig. 3. The architecture of Convolution-Enhanced GRU (CGRU) for capturing short-term dependencies.

computational process is expressed as follows:

$$\begin{aligned} z_{t_l} &= \phi(W_z h_{CGRU}^{(t_l-1)} + c_{t_l} + b_z), \\ r_{t_l} &= \phi(W_r h_{CGRU}^{(t_l-1)} + c_{t_l} + b_r), \\ \tilde{h}_{CGRU}^{(t_l)} &= \tanh(W(r_{t_l} \odot h_{CGRU}^{(t_l-1)} + c_{t_l} + b)), \\ h_{CGRU}^{(t_l)} &= (1 - z_{t_l}) \odot h_{CGRU}^{(t_l-1)} + z_{t_l} \odot \tilde{h}_{CGRU}^{(t_l)} \end{aligned} \quad (18)$$

where z_{t_l} and r_{t_l} represent the outputs of the update and reset gates, respectively, with $\phi(\cdot)$ denoting the sigmoid activation. The hidden state at time t_l , $h_{CGRU}^{(t_l)}$, is updated using the candidate state $\tilde{h}_{CGRU}^{(t_l)}$, which combines the current input c_{t_l} and the previous hidden state $h_{CGRU}^{(t_l-1)}$ via element-wise multiplication. The matrices W_z , W_r and W are learnable weights, with corresponding bias terms b_z , b_r and b . The output $y_{CGRU}^{(t_l)}$ at each time step is the current hidden state $h_{CGRU}^{(t_l)}$. The gating mechanisms enable the CGRU to selectively retain historical information and incorporate new data, making it particularly effective at modeling short-term temporal dependencies, such as those in fine robotic operations.

Final Output: The MDFB module combines global contextual information from the BDGM and short-term dependencies from the CGRU. The BDGM produces a global representation $y_{BDGM}^{(t_l)}$, while the CGRU captures local features through $y_{CGRU}^{(t_l)}$. These outputs are fused using a weighted sum:

$$y_{MDFB}^{(t_l)} = p_1 \cdot y_{BDGM}^{(t_l)} + p_2 \cdot y_{CGRU}^{(t_l)} \quad (19)$$

where p_1 and p_2 are learnable weights that balance the contributions of both modules. This fusion strategy effectively integrates long-term and short-term dependencies, providing a comprehensive representation for action modeling tasks.

4.3. Improved denoising network architecture

As illustrated in the Fig. 1(a), the denoising network employs a multi-module stacked architecture, with the MDFB as its core component. This module is designed to capture features across different temporal scales, enhancing the network's ability to model both long-term and short-term dependencies. Additionally, the network incorporates the SwiGLU activation function to perform nonlinear feature mapping, further increasing its expressive capacity [39,40] (refer to Fig. 4 for the detailed structure of SwiGLU). Residual connections are applied following each MDFB and SwiGLU module to integrate the output back into the input, thereby improving model stability during training and effectively mitigating gradient vanishing issues. The complete denoising network is constructed by repeating these modules L times, leveraging multiscale dependency fusion and residual learning to enhance feature representation diversity and improve denoising performance for intricate sequence modeling.

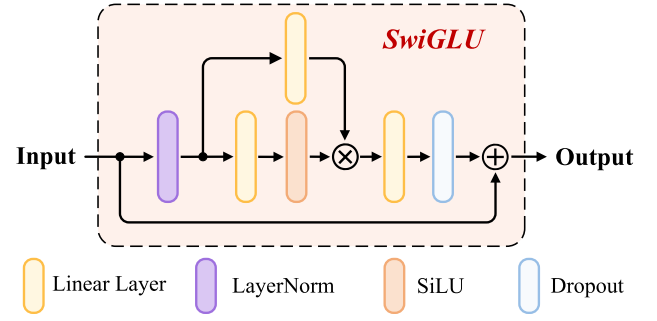


Fig. 4. The architecture of SwiGLU.

Algorithm 2: The Action Generation Process of Mamba2Diff.

Require: Current observation o , Goal g
Require: Score-Denoising Model $D_\theta(\tau, o, g, \sigma)$
Require: discrete time step $t_l \in \{0, \dots, N\}$
Require: Logarithmic noise decay function $\beta_t = \log(1 + t)$
Require: Noise schedule $\sigma_l = t_l$
1 Sample initial action $\tau_0 \sim \mathcal{N}(0, \sigma_0^2 I)$
2 **for** $l \in \{0, \dots, N-1\}$ **do**
3 $d_l \leftarrow \frac{\tau_l - D_\theta(\tau_l, o, g, \sigma_l)}{\sigma_l}$
4 $\beta_{t_l} \leftarrow \log(1 + t_l), \beta_{t_{l+1}} \leftarrow \log(1 + t_{l+1})$
5 $\delta_l \leftarrow \beta_{t_l} - \beta_{t_{l-1}}$
6 $\tau_{l+1} \leftarrow \left(\frac{t_{l+1}}{t_l}\right) \tau_l - (e^{-\delta_l} - 1) d_l$
7 **end**
Return: Final action τ_N

4.4. Efficient action generation process

To enable efficient and precise planning of complex robotic trajectories, we integrate the advantages of SGMs and the DDIM solver [28,41] to generate action sequences in an optimized manner. Algorithm 2 provides a summary of the action generation process in Mamba2Diff. Initially, the action sequence begins with sampling the initial action $\tau_0 \sim \mathcal{N}(0, \sigma_0^2 I)$ from a standard Gaussian distribution. Following the Eq. 5, noise is progressively removed by solving the reverse probability flow ODE to yield the final high-quality action sequence. During the numerical solving process, we approximate the score function with a pre-trained denoising network $D_\theta(\tau, o, g, \sigma(t))$, resulting in the following update formula:

$$\begin{aligned} \tau_{l+1} &= \left(\frac{t_{l+1}}{t_l}\right) \tau_l - (e^{-\delta_l} - 1) d_l, \\ d_l &= \frac{\tau_l - D_\theta(\tau_l, o, g, \sigma_l)}{\sigma_l}, \\ \delta_l &= \beta_{t_l} - \beta_{t_{l-1}} \end{aligned} \quad (20)$$

where τ_l represents the action sample at denoising step l , and σ_l represents the noise level at denoising step l , standardizing the denoising output. d_l indicates the update direction during each iteration, capturing the deviation between the current noisy sample and the estimated denoised sample. δ_l represents the step size, governed by the noise schedule, which adjusts the denoising step size. The noise schedule function β_t is typically defined logarithmically or exponentially to ensure smooth attenuation of the noise level across time steps.

This deterministic reverse diffusion process enables action sequence generation with only a few denoising steps (three iterations in our experiments were sufficient to achieve high success rate and time efficiency), ensuring both the quality and diversity of actions while significantly enhancing inference efficiency. In complex robotic tasks, this approach not

only generates smooth and precise action sequences but also effectively reduces computational cost.

5. Simulation experiments

This section provides a comprehensive evaluation of Mamba2Diff through multi-dimensional experiments conducted in simulated environments. The evaluation not only validates Mamba2Diff's effectiveness in capturing long-term dependencies and fusing multiscale dependencies, but also emphasizes its precision, consistency, and efficiency in long-horizon tasks. Specifically:

- **Precision** measures how accurately Mamba2Diff generates action sequences that align with the intended goals.
- **Consistency** evaluates Mamba2Diff's ability to maintain stable performance across extended task sequences, ensuring high-quality execution throughout.
- **Efficiency** assesses the number of steps required for the robot to complete long-horizon tasks, with fewer steps indicating better optimization in action planning.

5.1. Experimental setup

Baselines: In goal-conditioned behavior modeling, Mamba2Diff is compared with current SOTA methods, categorized into diffusion-based and non-diffusion-based approaches. Non-diffusion-based methods include Goal-Conditioned Behavior Cloning (GCBC) [24], Latent Motor Plans (LMP) [42], Conditional Implicit Behavior Cloning (C-IBC) [43], Conditional-Behavior Transformer (C-BeT) [25], and Vector-Quantized Behavior Transformer (VQ-BeT) [44]. Diffusion-based methods, by contrast, primarily include CX-Diff [26], C-BESO [19], and CFG-BESO [19]. Experimental results for GCBC, LMP, C-IBC, C-BeT, CX-Diff, C-BESO, and CFG-BESO in the Block-Push and Relay Kitchen environments are reported from [19], while those for VQ-BeT in these environments are reported from [44]. Furthermore, following the protocol detailed in [44], we constructed the Multimodal Ant environment, successfully reproducing reward and result metrics for GCBC, LMP, C-IBC, C-BeT, CX-Diff, C-BESO, and CFG-BESO. To ensure fairness in evaluation across all methods, we maintained consistency in general hyperparameters (e.g., layer size and depth) while adjusting method-specific hyperparameters as needed [19].

We evaluated Mamba2Diff against baseline methods using three simulation benchmarks, as illustrated in Fig. 5 and detailed in Table 1.

Multimodal Block-Push [43]: This environment assesses the policy network's ability to fit multimodal action distributions by requiring the robot to push two differently colored blocks into two matching colored squares in a random sequence. The environment consists of a 6-DoF

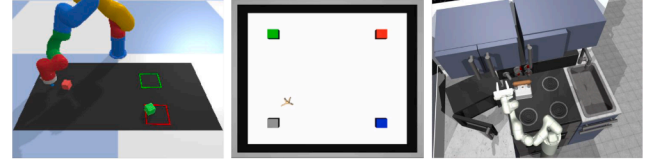


Fig. 5. Simulation environments for evaluating the performance of Mamba2Diff and baselines: Multimodal Block-push(left); Multimodal Ant (middle); Relay Kitchen (right).

XArm robotic arm, two blocks (red and green), and two target squares (also red and green). To enhance the original dataset, we incorporated an additional dataset that includes goal annotations for expert trajectories [45]. This supplementary dataset comprises 1000 expert trajectories generated by a scripted oracle, referencing ground-truth state information and ensuring randomized initial positions during data generation.

Multimodal Ant [46]: This environment features a MuJoCo Ant robot and four target regions, each marked with a distinct color (grey, blue, red, and green) at the four corners of the map. The expert demonstration dataset includes multiple trajectories showing the robot reaching these four target regions in varied sequences.

Franka Relay Kitchen [23]: Relay Kitchen is a simulated kitchen environment where a 9-DoF Panda robot is required to complete any 4 out of 7 possible tasks in random order. The dataset comprises 566 human demonstrations collected using VR controllers, each involving a unique randomized set of tasks among the 7 available, along with predefined goal states [45].

5.2. Evaluation metrics

The performance of each algorithm is measured by the following three metrics:

Result: This metric evaluates the extent to which the expected tasks are completed in each test round. Specifically, it assesses the deterministic decision-making ability of the policy network, i.e., whether the network parameters can be effectively tuned based on training data to make accurate decisions across similar task scenarios. We followed the parameter settings from [45]. As illustrated in Fig. 6, for the Multimodal Block-Push environment, 0.5 credit are awarded when a block is pushed into the intended square, with a maximum score of 1.0. For Multimodal Ant, the agent earns 1.0 credit each time it successfully reaches a target region, with a maximum achievable score of 4.0. In the Relay Kitchen environment, each completed task is scored at 1.0 credit, up to a maximum of 4.0 credits.

Reward: This metric assesses the number of tasks successfully completed during each testing round, extending beyond predefined tasks to

Table 1

All interactable objects, predefined tasks and steps of expert trajectories in three simulation environments.

Simulation Environments	Interactable Objects	Pre-defined Tasks	Steps of Expert Trajectories
Multimodal Block-Push	red block	pushing the red block into red square	272 ± 28
		pushing the red block into green square	
	green block	pushing the green block into red square	
		pushing the green block into green square	
Multimodal Ant	ant robot	reaching the grey region reaching the blue region reaching the red region reaching the green region	284 ± 16
Franka Relay Kitchen	microwave	turning on the microwave	262 ± 18
	kettle	moving the kettle	
	light switch	turning on the light switch	
	slide cabinet	opening the slide cabinet	
	hinge cabinet	opening the hinge cabinet	
	top burner bottom burner	turning the top burner turning the bottom burner	

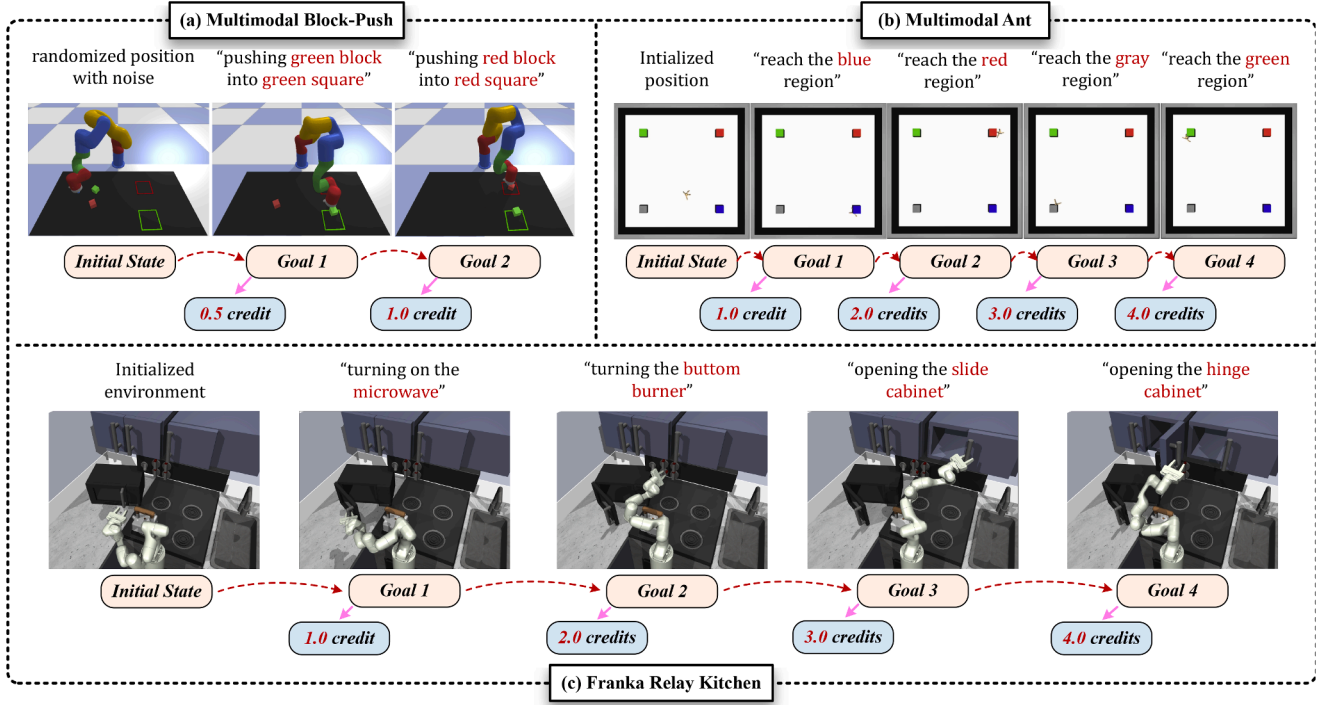


Fig. 6. Examples of the corresponding credits obtained by completing pre-defined tasks in three simulation environments: (a) Multimodal Block-Push, (b) Multimodal Ant, (c) Relay Kitchen.

evaluate the generalization of the policy network. Specifically, it examines whether the network can adapt to inputs beyond the scope of pre-defined tasks, which is crucial for applications in complex or dynamic environments. In Multimodal Block-Push and Multimodal Ant, both simulation environments where the number of predefined tasks aligns with the number of goals. For Multimodal Block-Push and Multimodal Ant, the maximum reward and result scores are 1.0 and 4.0, respectively. However, in the Relay Kitchen environment, the number of pre-defined tasks exceeds the number of goals, allowing for additional tasks to be completed beyond the target tasks during testing. Consequently, the maximum achievable reward score can exceed 4.0, with a maximum of 5.0 observed in our experiments.

Steps: This metric quantifies the number of action steps required for a robot to complete a long-horizon task. In the Multimodal Block-Push and Multimodal Ant environments, the maximum step count is 300, whereas for the Relay Kitchen environment, it is 280. If the step count exceeds these thresholds, the task is considered to have timed out and deemed incomplete.

5.3. Overall performance evaluation

As shown in Table 2, we conducted an overall evaluation of Mamba2Diff's performance across three simulation environments:

Block-Push, Multimodal Ant, and Relay Kitchen, focusing on precision and efficiency, with hyperparameters listed in Table 3. Mamba2Diff consistently outperformed non-diffusion-based methods, achieving higher scores across all environments. It is worth noting that VQ-BeT, a non-diffusion-based method, achieved competitive scores across the three environments, reflecting its ability to effectively capture multimodal action distributions through its hierarchical vector quantization module. Besides, in comparison with diffusion-based methods, Mamba2Diff consistently achieved the highest scores across all tested environments. To statistically validate these performance differences, we employed Analysis of Variance (ANOVA), a method used to assess whether there are statistically significant differences between the means of multiple groups. Specially, in the Relay Kitchen environment, it reached an average reward of 4.02 and result of 3.84, slightly surpassing C-BESO's 3.98 [$F(1, 198) = 6.89, p < .01$] and 3.75 [$F(1, 198) = 7.21, p < .01$]. In the Multimodal Ant environment, Mamba2Diff scored an average reward of 3.39 and result of 3.31, outperforming C-BESO, which scored 2.96 [$F(1, 198) = 8.15, p < .01$] and 3.29 [$F(1, 198) = 8.67, p < .01$], respectively. As for the Steps metric, it can be observed that Mamba2Diff completes the tasks with fewer action steps across all three environments, further emphasizing its ability to execute tasks with higher efficiency while maintaining high success rates.

Table 2

The calculated mean and standard deviation results over 10 seeds with 100 runs each on the Block-Push, Multimodal Ant and Relay Kitchen environment.

Method	Metrics	GCBC	C-IBC	LMP	C-BeT	VQ-BeT	CX-Diff	C-BESO	CFG-BESO	Mamba2Diff
Block-Push	Reward ↑	0.13 ± 0.04	0.46 ± 0.06	0.04 ± 0.03	0.91 ± 0.03	0.93 ± 0.03	0.90 ± 0.02	0.96 ± 0.02	0.97 ± 0.02	0.97 ± 0.01
	Result ↑	0.13 ± 0.04	0.29 ± 0.10	0.04 ± 0.03	0.87 ± 0.07	0.87 ± 0.01	0.90 ± 0.04	0.93 ± 0.02	0.88 ± 0.04	0.95 ± 0.01
	Steps ↓	237 ± 28	225 ± 23	254 ± 19	167 ± 19	133 ± 14	178 ± 14	145 ± 13	154 ± 14	124 ± 12
Multimodal Ant	Reward ↑	2.21 ± 0.19	0.42 ± 0.09	1.27 ± 0.16	2.43 ± 0.16	3.33 ± 0.06	2.96 ± 0.14	3.29 ± 0.06	3.18 ± 0.07	3.39 ± 0.04
	Result ↑	2.16 ± 0.18	0.37 ± 0.11	1.2 ± 0.18	2.34 ± 0.17	3.22 ± 0.06	2.85 ± 0.12	3.17 ± 0.04	2.98 ± 0.05	3.31 ± 0.05
	Steps ↓	260 ± 25	253 ± 23	272 ± 18	225 ± 18	193 ± 19	221 ± 21	198 ± 16	207 ± 18	182 ± 17
Relay Kitchen	Reward ↑	2.65 ± 0.25	0.50 ± 0.09	1.45 ± 0.22	2.73 ± 0.28	3.96 ± 0.06	3.64 ± 0.14	3.98 ± 0.07	3.98 ± 0.07	4.02 ± 0.04
	Result ↑	2.57 ± 0.26	0.45 ± 0.08	1.41 ± 0.22	2.69 ± 0.28	3.78 ± 0.06	3.35 ± 0.15	3.75 ± 0.08	3.47 ± 0.08	3.84 ± 0.05
	Steps ↓	250 ± 17	248 ± 18	257 ± 13	247 ± 26	231 ± 26	242 ± 19	224 ± 28	234 ± 30	216 ± 26

Table 3

Key hyperparameter settings across three simulation environments.

Hyperparameter	Block-Push	Multimodal Ant	Relay Kitchen
Hidden Dimension	240	512	360
Hidden Layers	4	6	6
Window size	5	5	5
Goal window size	1	1	2
Number Heads	12	6	6
Head Dimension	20	64	30
Dynamic State	64	64	64
Conv Dimension	4	4	4
Expand Factor	2	2	2
Train steps	40000	60000	40000
Learning Rate	$1e^{-4}$	$1e^{-5}$	$1e^{-4}$
Denoising Steps	3	3	3
σ_{data}	0.5	0.5	0.5
σ_{min}	0.05	0.005	0.005
σ_{max}	1	1	1
Optimizer	Adam	Adam	Adam
EMA	True	True	True
Sampler Type	DDIM	DDIM	DDIM
Noise distribution	Log-logistic	Log-logistic	Log-logistic
Noise scheduler	exp	exp	exp
Batch Size	256	256	256
Train Steps in Thousand	60	40	56
Primitive Action Range	$[-1, 1]$	$[-1, 1]$	$[-1, 1]$
Noise Range	$\{0.005, 1\}$	$\{0.005, 1\}$	$\{0.005, 1\}$

These results underscore Mamba2Diff's effectiveness in integrating both long-term and short-term dependencies, primarily facilitated by the MDFB module. This module enhances the model's capability to process complex action generation tasks across varying sequence lengths. By seamlessly fusing information from different temporal scales, the MDFB module allows Mamba2Diff to adaptively manage immediate and extended task requirements, maintaining precision in dynamic environments.

5.4. Goal-oriented action generation

To further analyze Mamba2Diff's capability in goal-oriented action generation, we evaluated its ability to extract goal information and generate actions that align with the specified objectives, which is essential for assessing the model's precision in achieving the goals. We compared it with leading baselines (CX-Diff, C-BESO, and VQ-BeT) within the Relay Kitchen environment (as shown in Figure Fig. 7). The details of the comparison tasks can be found in Table 1. Results indicate that Mamba2Diff consistently achieved higher average success rates across all tasks, particularly excelling in complex, goal-oriented scenarios. For

example, Mamba2Diff achieved a 100 % average success rate in tasks requiring fine-grained control, such as "turning on the microwave," "opening the hinge cabinet," and "turning the bottom burners", outperforming C-BESO and VQ-BeT, which attained average success rates ranging from 96 % to 98 %. Additionally, in challenging tasks such as "turning on the light switch," Mamba2Diff still achieved a success rate of 83 %, compared to 79 % [$F(1, 198) = 4.02, p < .05$] for VQ-BeT and 74 % [$F(1, 198) = 7.85, p < .01$] for C-BESO.

These findings suggest that the impressive performance of Mamba2Diff in executing goal-oriented tasks may be attributed to its innovative design features. Specifically, the BDGM module, which effectively manages information flow through the AAG, allows the model to extract global contextual information. This design enables Mamba2Diff to capture dynamic information relevant to task objectives, supporting effective decision-making and action modeling. By optimizing information flow and enhancing global information acquisition, the BDGM module significantly boosts Mamba2Diff's accuracy, facilitating its superior performance in complex action generation scenarios.

5.5. Long-horizon action modeling

Building on the experiments in Section 5.3, this section analyzes Mamba2Diff's performance in long-horizon tasks from quantitative and qualitative perspectives, using three leading algorithms in Table 2, including CX-Diff, C-BESO, and VQ-BeT. In the quantitative analysis, we evaluate success rates at different stages of long-horizon tasks in the Relay Kitchen and Multimodal Ant environments, chosen for their multi-step task structures. Additionally, statistical analysis of execution step distributions is conducted to assess the model's efficiency and consistency. In the qualitative analysis, we compare Mamba2Diff with the baselines on sequence memory and precision in fine-grained tasks.

Quantitative Analysis: As presented in Fig. 8(a) and Fig. 8(b), Mamba2Diff consistently outperformed other baselines across all task stages in both the Relay Kitchen and Multimodal Ant environments, demonstrating a clear advantage in long-horizon action modeling. In the Relay Kitchen environment, for the fourth predefined target task, Mamba2Diff achieved an average success rate of 93 %, significantly higher than C-BESO's 84 % [$F(1, 198) = 7.18, p < .01$] and VQ-BeT's 89 % [$F(1, 198) = 6.79, p < .01$]. For the fifth extra, non-predefined task, it maintained an average success rate of 21 %, surpassing C-BESO's 14 % and VQ-BeT's 16 %, highlighting the model's generalization ability. Similarly, in the Multimodal Ant environment, Mamba2Diff demonstrated strong performance, particularly in the later stages of the task sequence, achieving average success rates of 87 % and 49 % for the third and

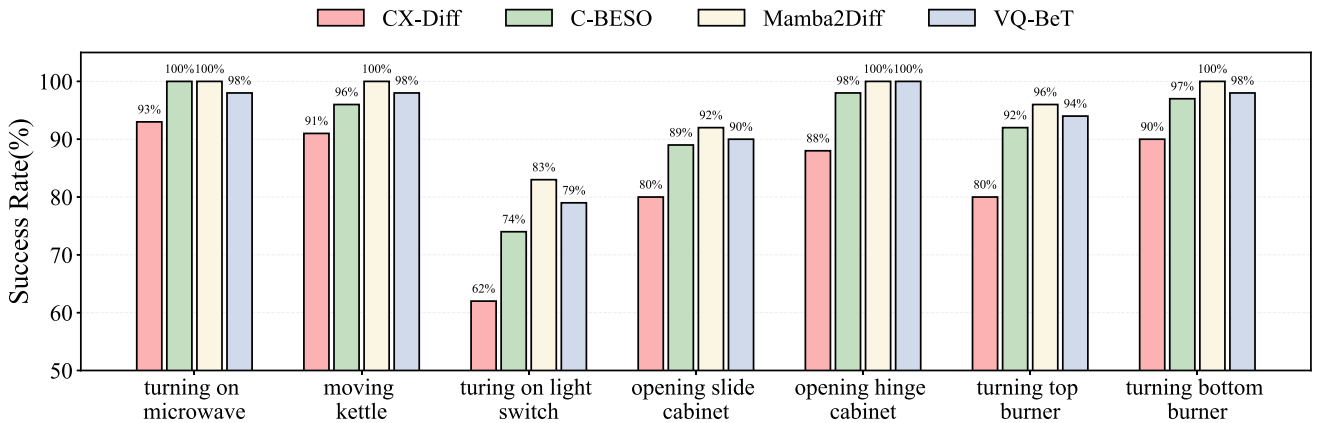


Fig. 7. Comparison of success rates for Mamba2Diff and baseline methods (CX-Diff, C-BESO, VQ-BeT) on single hard tasks in the Relay Kitchen environment, with mean success rates calculated over 10 seeds and 100 runs per seed.

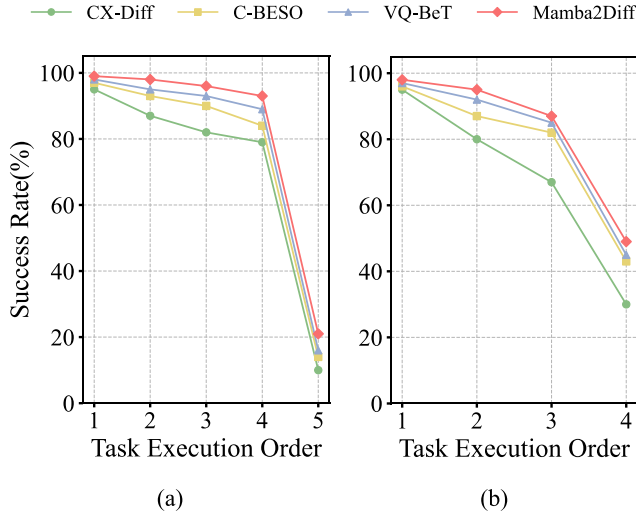


Fig. 8. Success rates for sequential tasks performed by Mamba2Diff and baseline models in long-horizon scenarios within the (a) Relay Kitchen and (b) Multimodal Ant environments, with mean success rates calculated over 10 seeds and 100 runs per seed.

fourth tasks, respectively, far surpassing VQ-BeT's averages of 30% [$F(1, 198) = 11.24, p < .001$] and 45% [$F(1, 198) = 8.67, p < .01$].

In addition, as demonstrated by the statistical comparison of execution steps among different algorithms in Fig. 10, Mamba2Diff not only achieves superior success rates in long-horizon tasks but also demonstrates greater efficiency in modeling action sequences. Specifically, Mamba2Diff consistently yields significantly lower mean and median numbers of execution steps across three simulation environments, indicating that the generated action sequences are more compact and ef-

ficient. Furthermore, the lower frequency of timeout outliers observed with Mamba2Diff implies fewer redundant or erroneous actions, highlighting enhanced consistency during task execution. Such efficiency and consistency inherently reflect the model's improved capability in capturing and utilizing global contextual information to coordinate sequential actions over extended horizons.

Qualitative Analysis: Furthermore, to visually illustrate the advantages of Mamba2Diff in long-horizon action generation, we selected a representative task sequence in the Relay Kitchen environment, comprising four sequential steps: "turning on the bottom burner", "turning on the top burner", "opening the slide cabinet", and "opening the hinge cabinet" (as shown in Fig. 9). This experiment compares Mamba2Diff's performance with three baseline algorithms: CX-Diff, C-BESO, and VQ-BeT. CX-Diff encountered multiple execution errors. It initially failed to align the gripper correctly to turn on the bottom burner, attempted an unnecessary action by activating the microwave, and ultimately failed to open the hinge cabinet due to poor positioning. C-BESO successfully completed the first step but failed in the second task due to an insufficient switch rotation angle and encountered issues in the fourth step with an inadequate gripping angle. VQ-BeT performed well in the first three steps but was unable to grasp the hinge cabinet handle correctly in the fourth step, preventing task completion. In contrast, Mamba2Diff successfully completed all four steps with high accuracy, demonstrating superior precision in action execution and enhanced sequential memory for long-horizon tasks.

This sustained high performance across diverse environments and complex multimodal tasks underscores Mamba2Diff's significant advantage in maintaining consistency in long-horizon action generation while achieving precise control over baseline models. The MDFB module plays a critical role in this success by ensuring robust global information modeling within complex, long-horizon tasks. By effectively integrating multiscale dependencies and adeptly extracting and processing local features, Mamba2Diff enhances execution accuracy and consistency in intricate, long-sequence tasks, making it particularly well-suited for multi-step robotic actions.

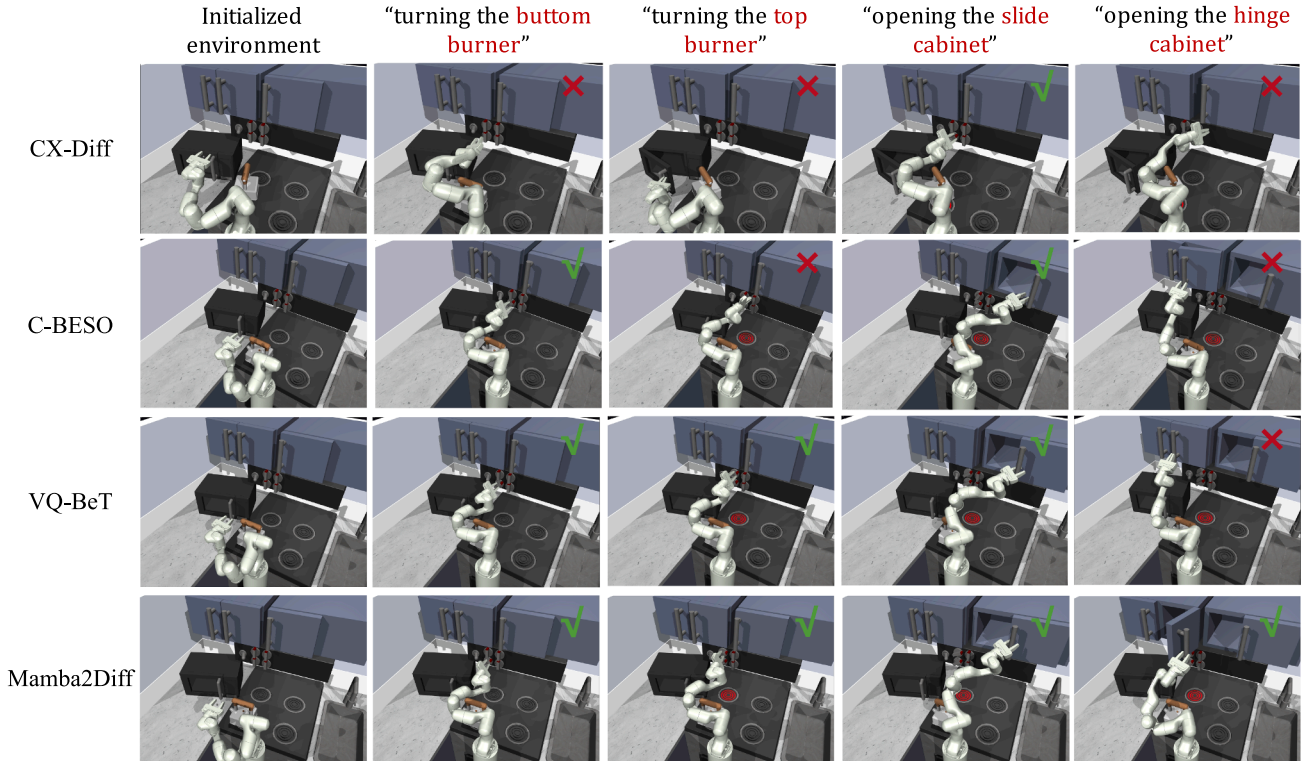


Fig. 9. Performance comparison of Mamba2Diff and baseline methods on a representative long-horizon action execution case in the Relay Kitchen environment.

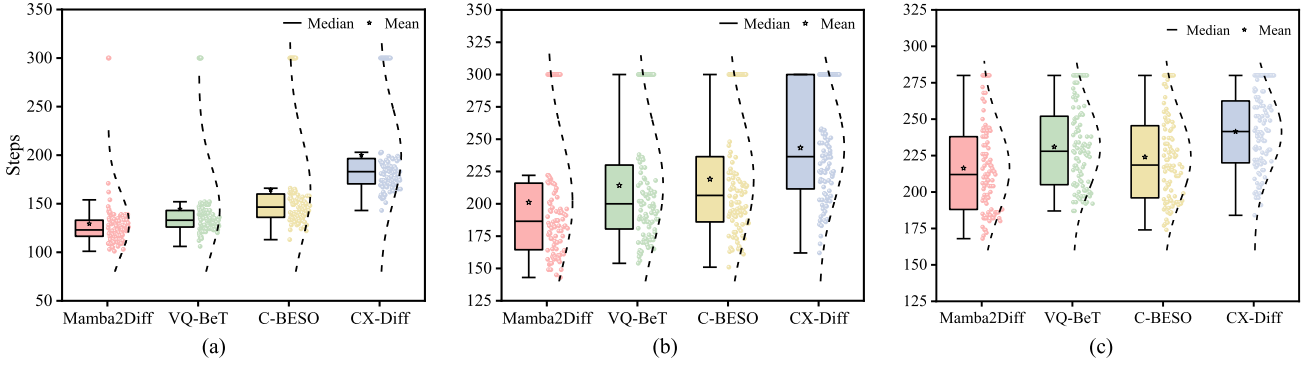


Fig. 10. Comparison of execution steps distributions between Mamba2Diff and baseline methods across three virtual environments: Multimodal Block-push (a); Multimodal Ant (b); Relay Kitchen (c), with mean results calculated over 10 seeds with 100 runs each.

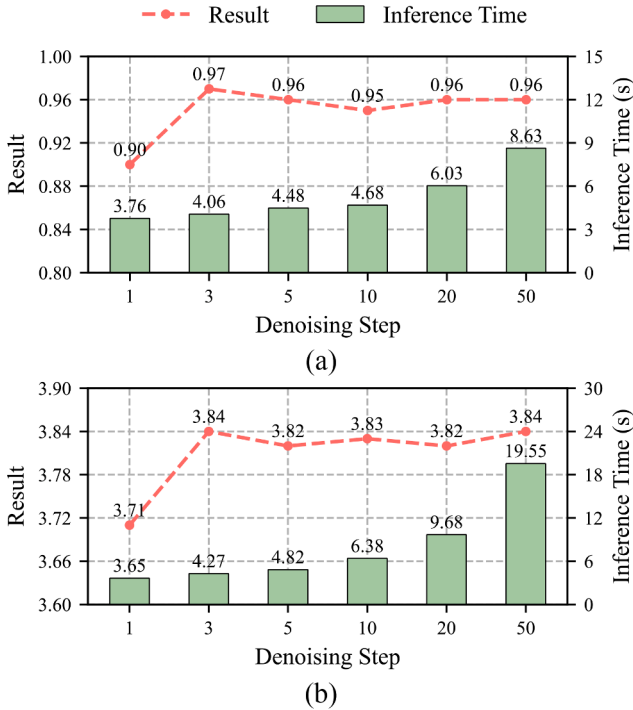


Fig. 11. Analysis of result and inference time (both averaged over 10 seeds with 20 runs each) variation over different iterations of Mamba2Diff in two simulation environments: (a) Block-Push and (b) Relay Kitchen.

5.6. Impact of denoising step setting

The configuration of denoising steps is critical in diffusion models as it directly influences task success rates and inference times. To determine the optimal setting, experiments were conducted in two simulation environments, Block-Push and Relay Kitchen, assessing how varying denoising steps l (with values: 1, 3, 5, 10, 20, 50) influence performance metrics. As illustrated in Fig. 11, the x-axis denotes the number of diffusion model iterations, while the y-axes show the result metric (left) and the average inference time per full task sequence (right).

In the Block-Push environment, increasing l initially enhances the result metric, peaking at an average value of 0.97 at $l = 3$, after which performance slightly declines and stabilizes at higher values. For $l \geq 3$, the result metric remains relatively steady. In contrast, inference time consistently increases with higher l , from an average of 3.76 seconds at $l = 1$ to 8.63 seconds at $l = 50$. A similar trend was observed in the Relay Kitchen environment, where the result metric reached its highest aver-

age value of 3.84 at $l = 3$ before a gradual stabilization at higher steps. In this environment, inference time increased more notably, ranging from an average of 3.65 seconds at $l = 1$ to 19.55 seconds at $l = 50$, highlighting the higher computational cost associated with larger denoising steps.

Overall, these results suggest that a denoising step of $l = 3$ achieves the best balance for the Mamba2Diff algorithm in both environments, providing an optimal trade-off between task success rate and inference time.

5.7. Computational cost and real-time efficiency analysis

Unlike traditional diffusion models such as CX-Diff and C-BESO, which rely on MLP or Transformer-based architectures for their denoising networks, Mamba2Diff adopts a novel bidirectional Mamba2 structure, offering significant advantages in computational cost and real-time efficiency. The bidirectional flow of information in our proposed MDFB modules enables more efficient long-range dependency modeling and parameter utilization, thereby accelerating the convergence of the denoising process. As shown in Table 4, compared to diffusion models (CX-Diff, C-BESO), Mamba2Diff reduces the number of parameters by 28%–45% (e.g., in the Multimodal Ant environment, CX-Diff has 1.4 times more parameters), while achieving 1.9–8.1 times faster inference speeds (e.g., in the Multimodal Ant environment, CX-Diff takes 8 times longer), demonstrating its dual advantages in both computational cost and real-time performance. Compared to non-diffusion lightweight models (VQ-BiT), Mamba2Diff achieves comparable or even lower parameter counts, while maintaining or even surpassing inference speed (e.g., in the Relay Kitchen environment). Additionally, Mamba2Diff retains the unique ability of diffusion models to generate fine-grained action sequences, making it more competitive in balancing real-time performance and task precision. All experiments were conducted on a hardware system equipped with an Intel Core i5-14600KF CPU, NVIDIA GeForce RTX 4090 D Graphics Processing Unit (GPU), and 32 GB of RAM.

5.8. Ablation study

Core Network Ablations: To identify the most suitable architecture for long-sequence modeling tasks, as shown in Fig. 12, we compared four network designs: CNN, Transformer, Mamba-V1, and Mamba-V2, evaluating their performance using two main metrics: reward and result. In the experiments, only the core network architecture was replaced, while the overall framework remained unchanged. The experimental results indicate that Mamba-V2 outperformed all other models across the three environments, particularly in capturing long-range dependencies. In the Kitchen environment, Mamba-V2 achieved a result score of 3.84, which represents an improvement of 73.73% over CNN

Table 4

Params and Inference Times of Single Step for Mamba2Diff and baselines in three simulation environments (both averaged over 10 seeds with 20 runs each).

Method	Params(M)			Inference Times of Single Step (ms)		
	Block-push	Multimodal Ant	Relay Kitchen	Block-push	Multimodal Ant	Relay Kitchen
CX-Diff	3.7	12.5	5.1	16.2	108.6	13.6
C-BESO	2.8	9.4	3.7	9.0	25.9	8.6
VQ-BeT	2.6	9.2	3.2	8.3	15.1	8.4
Mamba2Diff	2.4	8.9	2.8	7.7	13.4	7.9

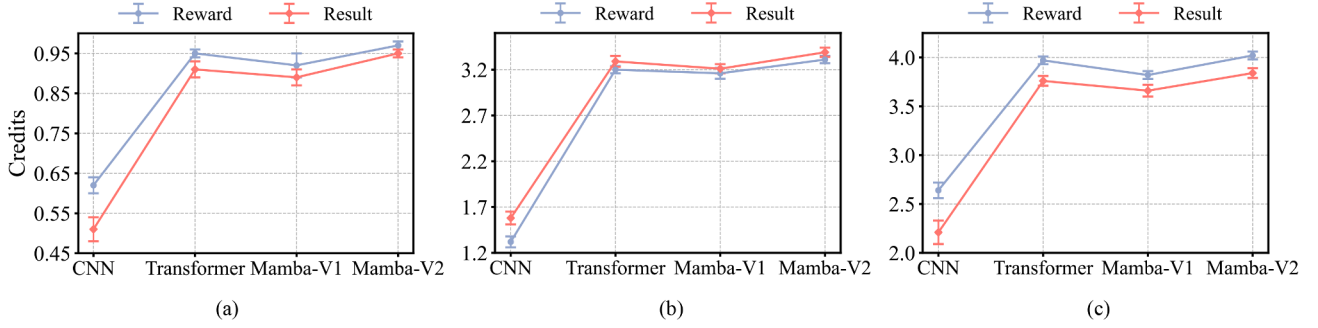


Fig. 12. Performance comparison of four types of core network architectures across 3 simulation Environments: Multimodal Block-push(a); Multimodal Ant (b); Relay Kitchen (c), with mean results calculated over 10 seeds with 20 runs each.

(result: 2.21) [$F(1, 198) = 11.78$, $p < .001$], 2.13 % over Transformer (result: 3.76) [$F(1, 198) = 6.81$, $p < .01$], and 4.93 % over Mamba-V1 (result: 3.66) [$F(1, 198) = 7.43$, $p < .01$]. In the Ant environment, Mamba2's result of 3.39 exceeded CNN's result of 2.74 [$F(1, 198) = 6.91$, $p < .01$], Transformer's result of 3.25 [$F(1, 198) = 5.32$, $p < .05$], and Mamba-V1's result of 3.15 [$F(1, 198) = 8.21$, $p < .01$]. Additionally, Mamba-V2 also demonstrated superior performance in the Block-Push environment compared to the other models. The experimental results reveal that CNN's local perception mechanism is unable to effectively capture long-range temporal dependencies, resulting in poorer performance. Although Transformer employs a self-attention mechanism that enables the capture of global dependencies, its efficiency in handling cross-time-step dependencies in long-sequence tasks is lower, which affects its ability to capture long-term dependencies with sufficient precision, leading to inferior performance compared to Mamba-V2. While Mamba-V1 is capable of capturing long-sequence dependencies, its simplified architecture and memory mechanism fail to address long-term dependency issues effectively in complex tasks, resulting in performance inferior to that of Mamba-V2. Based on the experimental results in the Block-Push, Kitchen, and Ant environments, Mamba-V2 demonstrated exceptional performance in long-sequence modeling tasks and was ultimately selected as the core network for the denoising network in our proposed framework.

Module-Level Ablations: We conducted ablation studies in the Block-Push and Relay Kitchen simulation environments to evaluate the impact of each Mamba2Diff component on task performance, particularly on the reward and result metrics. While reward measures overall task completion, the result metric is especially crucial as it assesses the policy network's deterministic decision-making accuracy based on learned data across similar tasks. Consequently, our analysis focuses primarily on how individual components influence the result metric.

As shown in Table 5, removing SwiGLU reduced the result metric slightly, with average scores decreasing from 0.95 to 0.93 [$F(1, 198) = 6.86$, $p < .01$] in Block-Push and from 3.84 to 3.81 [$F(1, 198) = 6.83$, $p < .01$] in Relay Kitchen. SwiGLU plays a key role in enhancing non-linear mapping, enabling the model to capture complex features critical for precision. The AAG module had a more pronounced effect, particularly in long-horizon tasks, where its removal led to a decrease in average results from 0.95 to 0.90 [$F(1, 198) = 5.78$, $p < .05$]

Table 5

ablation study demonstrating the impact of model components on mean and standard deviation across 10 seeds and 20 runs in Block-Push and Relay Kitchen environments.

	Block-Push		Relay Kitchen	
	Reward	Result	Reward	Result
w/o SwiGLU	0.95 ± 0.01	0.93 ± 0.02	3.93 ± 0.04	3.81 ± 0.03
w/o AAG	0.92 ± 0.04	0.90 ± 0.02	3.75 ± 0.04	3.71 ± 0.05
w/o CGRU	0.87 ± 0.01	0.83 ± 0.03	3.81 ± 0.06	3.76 ± 0.05
w/o AAG + CGRU	0.84 ± 0.04	0.81 ± 0.03	3.72 ± 0.06	3.68 ± 0.04
w/o AAG + CGRU + SwiGLU	0.82 ± 0.03	0.78 ± 0.04	3.64 ± 0.05	3.62 ± 0.05
Mamba2Diff	0.97 ± 0.01	0.95 ± 0.01	4.02 ± 0.04	3.84 ± 0.05

in Block-Push and from 3.84 to 3.71 [$F(1, 198) = 8.13$, $p < .01$] in Relay Kitchen. This highlights AAG's contribution to dynamically optimizing information flow, thus improving accuracy in sequential tasks. The CGRU module, essential for short-sequence tasks, also showed a substantial impact: removing it led to a decrease in the result metric from an average of 0.95 to 0.83 [$F(1, 198) = 7.57$, $p < .01$] in Block-Push, underscoring its importance in capturing short-term dependencies. In Relay Kitchen, the result metric decreased modestly from 3.84 to 3.76 [$F(1, 198) = 7.01$, $p < .01$], indicating CGRU's role in processing local information even in long-sequence tasks. With both AAG and CGRU removed, the result metric declined further to averages of 0.81 in Block-Push and 3.68 in Relay Kitchen, underscoring their complementary roles in managing long- and short-term dependencies. A more significant performance drop was observed when SwiGLU, AAG, and CGRU were removed simultaneously, the reward and result metrics reached the lowest values among all the test environments. This outcome serves as an internal baseline to clearly demonstrate the effectiveness of the components introduced in our proposed method-SwiGLU, AAG, and CGRU each contribute uniquely to enhancing the model's performance. Ultimately, the complete Mamba2Diff model achieved the highest average result metrics across both environments, scoring 0.95 in Block-Push and 3.84 in Relay Kitchen. These findings confirm that the integrated components enable Mamba2Diff to maintain high-precision decision-making and task consistency, fulfilling its design objectives across diverse and challenging scenarios.

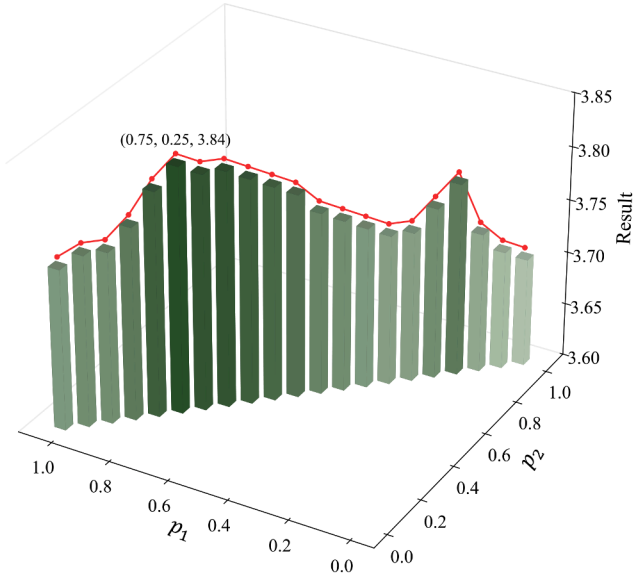


Fig. 13. Analysis of the impact of two learnable weights p_1 and p_2 on results in the MDFB module within the Relay Kitchen environment, where p_1 and p_2 control the BDGM and CGRU branches, respectively, with mean results calculated over 10 seeds with 20 runs each.

Learnable Weight Parameters: We analyzed the impact of the MDFB module's two control parameters, p_1 and p_2 , on model performance in the Relay Kitchen and Block-Push environments, aiming to validate the effectiveness of the BDGM and CGRU branches for distinct task requirements. To assess their influence, we manually set p_1 and p_2 values incrementally between 0 and 1 (by steps of 0.05), ensuring that $p_1 + p_2 = 1$. The result metric was prioritized, as it directly reflects decision accuracy in meeting task objectives. In the Relay Kitchen environment, as shown in Fig. 13, which requires effective long-horizon action modeling, increasing the BDGM branch weight p_1 led to improved performance. The highest average result of 3.84 was achieved at $p_1 \approx 0.75$, indicating that the BDGM module efficiently captures long-term dependencies within this range. However, further increases in p_1 resulted in a slight performance decline, suggesting that an excessive focus on long-term information may reduce the model's ability to capture short-term details effectively. This balance implies that an appropriately weighted p_1 is essential for optimized long-horizon action modeling. Conversely, as illustrated in Fig. 14, in the Block-Push environment, short-term dependencies are crucial for task success, with the CGRU branch being central to short-sequence actions. Optimal performance, with an average result of 0.95, was achieved when p_2 was around 0.65, highlighting CGRU's role in capturing short-term dependencies. Beyond this point, further increases in p_2 led to a slight reduction in performance, indicating that over-emphasis on short-term information can detract from the benefits of long-term contextual information. These findings underscore the MDFB's adaptability across varying task types: by dynamically adjusting p_1 and p_2 , the model effectively balances long-term and short-term dependencies, optimizing temporal information processing according to specific task demands.

Noise Sampler: In this ablation study, we evaluated several noise sampling algorithms in two simulation environments (Block-Push and Relay Kitchen) to determine if DDIM, the current sampler in our framework, is optimal. The algorithms examined include DPM [47], DDIM [41], UniPC [48], DPM-Solver++ [47], and DPM-Solver-v3 [49]. Each of these samplers has been widely applied in diffusion-based generation tasks and features distinct characteristics:

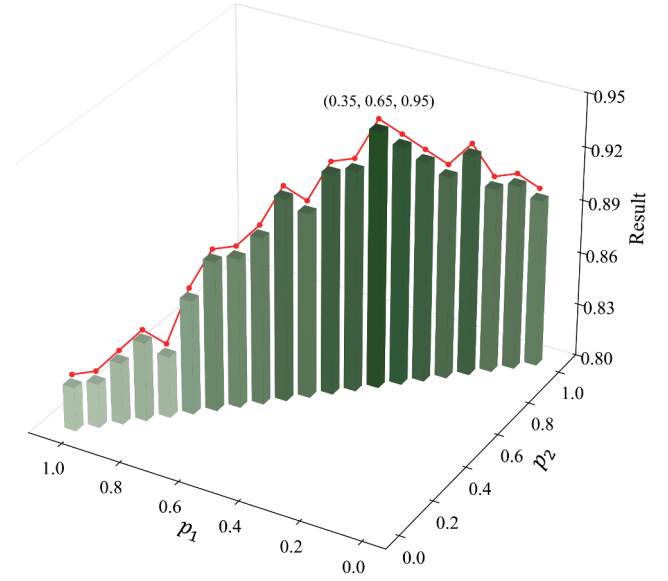


Fig. 14. Analysis of the impact of two learnable weights p_1 and p_2 on results in the MDFB module within the Block-Push environment, where p_1 and p_2 control the BDGM and CGRU branches, respectively, with mean results calculated over 10 seeds with 20 runs each.

- **DPM:** A second-order exponential ODE integrator used in the experiment tailored for efficient synthesis with minimal inference steps.
- **DDIM:** An enhanced diffusion sampling method employing a non-Markovian process for deterministic generation, achieving high-quality outputs with fewer steps.
- **UniPC:** A unified predictor-corrector framework for diffusion probabilistic models (DPMs) that combines a “Unified Predictor” and “Unified Corrector” to ensure high-order accuracy across variable sampling steps.
- **DPM-Solver++:** An ODE solver with data prediction parameterization and dynamic thresholding, which minimizes train-test distribution mismatch by applying a multistep, second-order approach for faster sampling.
- **DPM-Solver-v3:** An optimized ODE solver designed for faster sampling via reparameterization and a multistep predictor-corrector framework, significantly reducing the necessary steps.

Results are presented in Fig. 15. In the Block-Push environment, DDIM achieved the highest average scores for both reward and result metrics, with values of 0.97 and 0.95, respectively, slightly outperforming DPM++ (average reward of 0.96 [$F(1, 198) = 4.21, p < .05$] and result of 0.93 [$F(1, 198) = 3.93, p < .05$]) and DPM-Solver-v3 (average reward of 0.95 [$F(1, 198) = 4.76, p < .05$] and result of 0.92 [$F(1, 198) = 5.15, p < .05$]). This suggests that DDIM offers superior task completion quality and success rates. UniPC's slightly lower scores suggest limited adaptability in this environment. In the Relay Kitchen environment, DDIM also led in both reward and result scores, with average values of 4.02 and 3.84, respectively, followed closely by DPM++ (average reward of 3.94 [$F(1, 198) = 4.98, p < .05$] and result of 3.80 [$F(1, 198) = 5.36, p < .05$]) and DPM-Solver-v3 (average reward of 3.97 [$F(1, 198) = 4.93, p < .05$] and result of 3.79 [$F(1, 198) = 4.74, p < .05$]). The consistent high performance of DDIM across these metrics is largely due to its deterministic sampling approach, which maintains generation quality and task consistency even with fewer inference steps—a limitation observed in other samplers. Overall, DDIM demonstrates a superior balance between generation efficiency and quality in both environments.

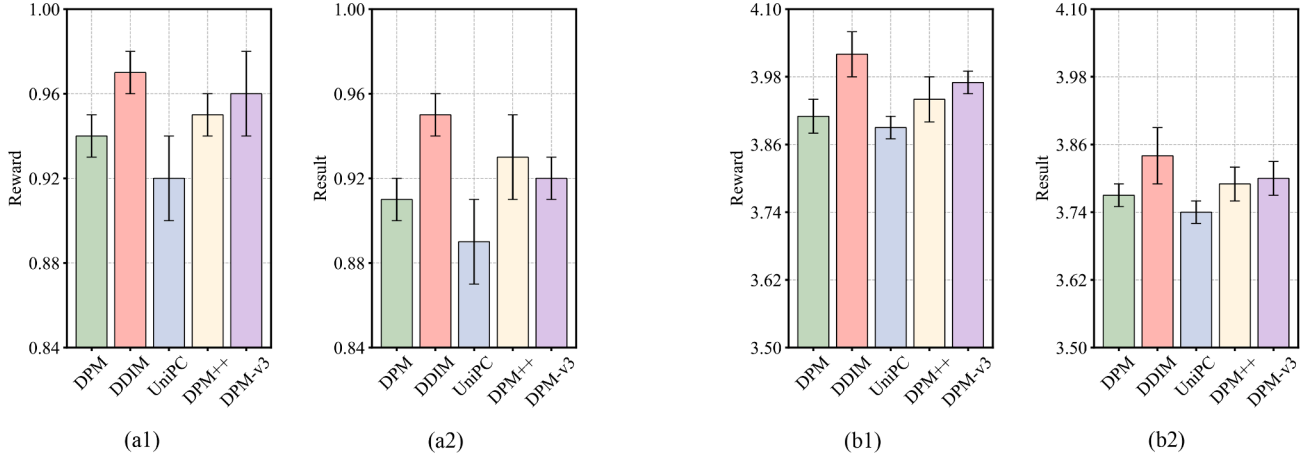


Fig. 15. The comparison of the effect of different noise sampling algorithm on the reward and result metric in two simulation environments, with mean and standard deviation of result and reward calculated over 10 seeds with 20 runs each. (a1-a2): Block-Push, (b1-b2): Relay Kitchen.

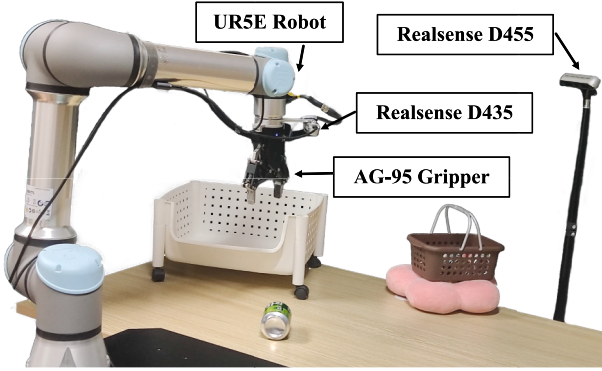


Fig. 16. Real-world robot platform setup.

6. Real-World experiments

In the previous sections, we validated the effectiveness of the proposed algorithm through experiments conducted in three distinct simulation environments. Building on these results, we constructed a real-world robotic platform (as shown in Fig. 16) and collected human demonstrations to assess the algorithm's performance in a practical setting.

Environment Setup and Data Collection: The primary robotic agent used in our real-world setup is a UR5E arm with a DaHuang AG-95 gripper attachment. Observational inputs for training the policy network come from two sources: a D435 RGB-D wrist camera mounted on the robotic arm, which captures precise information about the gripper's immediate workspace, and a D455 environment camera used to capture global scene information. Following the experimental protocol from [27], we used a Gello [50] to operate the robot remotely at a 30Hz frequency, collecting six degrees of freedom (6-DoF) human demonstrations. In total, we collected 576 trajectories across six different operators. As outlined in [51], we performed random geometric sampling immediately after each task's completion point to obtain goal images.

Scenarios Design: To evaluate the proposed algorithm's capability in long-horizon tasks, we designed two real-world scenarios (as shown in Fig. 17). In the first scenario, a can is scattered on the table, alongside a basket containing several cans and a shelf intended for final storage. The task requires the robot to first place the scattered can into the basket, lift the basket, and finally place it on the shelf. This scenario includes five phases, with the robot needing to execute four steps to complete the task. The second scenario features a bag of items and a hinge cabinet,

Table 6

Completion Counts (CC) of single task and Overall Success Rate (OSR) of all tasks for Mamba2Diff and three baseline methods in the Real-World Scenario I Experiment (20 runs each).

Method	Task 1	Task 2	Task 3	Task 4	OSR(%)
	CC	CC	CC	CC	
CX-Diff	18/20	16/20	14/20	11/20	73.75
C-BESO	20/20	17/20	16/20	14/20	83.75
VQ-BeT	20/20	18/20	17/20	15/20	87.5
Mamba2Diff	20/20	19/20	18/20	18/20	94.75

with the task requiring the robot to open the cabinet, place the bag inside, and then close the cabinet. Similar to the first scenario, this task also involves four sequential steps for completion.

Model Training and Evaluation: We reproduced the CX-Diff, C-BESO, and VQ-BeT algorithms on our real-world platform and used them as benchmarks to validate the advantages of our proposed Mamba2Diff in executing long-horizon tasks. Among the four methods, CX-Diff required the longest training time at approximately 4 hours, while the other algorithms each took about 2 h on an Nvidia RTX 4090 D GPU. During evaluation, each real-world scenario was repeated 20 times to mitigate random variation. To comprehensively assess the policy network's overall task completion capability across both scenarios and its sustained performance in long-horizon tasks, we introduced two evaluation metrics: Completion Count (CC) and Overall Success Rate (OSR).

Result Analysis: In real-world experiments, we compared Mamba2Diff with three baseline algorithms (CX-Diff, C-BESO, and VQ-BeT), recording each task's average completion count (CC) and overall success rate (OSR). The experimental results for two real-world scenarios are presented in Table 6 and Table 7. Mamba2Diff demonstrated superior performance, particularly in the later, more challenging stages of long-sequence tasks, achieving average OSRs of 94.75% and 91.25% in the first and second scenarios, respectively, significantly outperforming the baseline methods. Specially, in task 4 of the first scenario-a critical later-stage task-Mamba2Diff achieved an average CC of 18 out of 20 attempts, compared to 15, 14, and 11 for VQ-BeT, C-BESO, and CX-Diff, respectively. Similarly, in task 4 of the second scenario, Mamba2Diff averaged 17 completions out of 20 attempts, again surpassing the performance of the baseline algorithms. These findings underscore Mamba2Diff's enhanced stability and execution capability in later, more complex stages of long-sequence tasks, demonstrating its adaptability for multi-step planning and complex task execution.

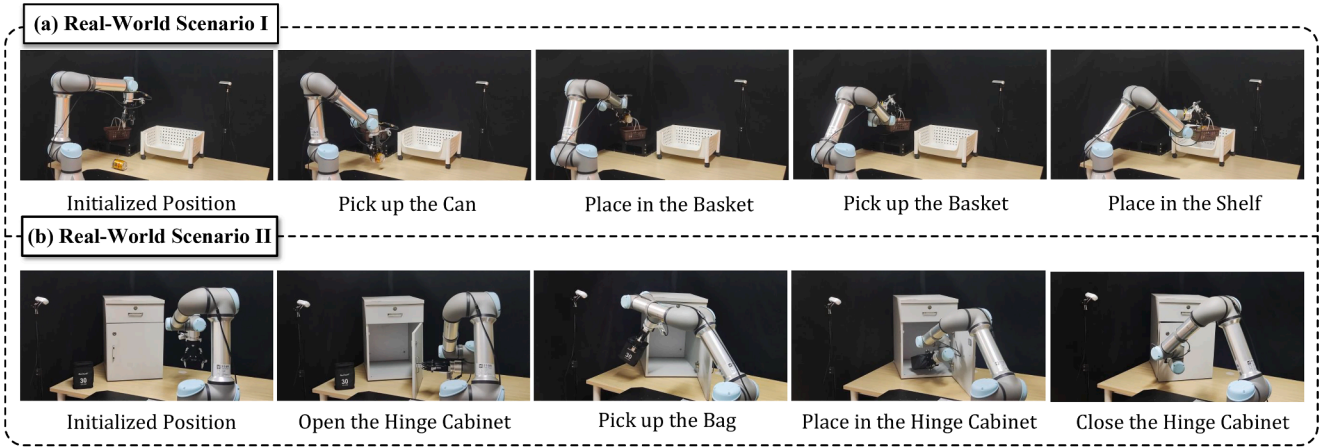


Fig. 17. Real-world scenarios designed to validate the effectiveness of our proposed algorithms.

Table 7

Completion Counts (CC) of single task and Overall Success Rate (OSR) of all tasks for our proposed algorithm and three baseline methods in the Real-World Scenario II Experiment (20 runs each).

Method	Task 1	Task 2	Task 3	Task 4	OSR(%)
	CC	CC	CC	CC	
CX-Diff	17/20	15/20	13/20	11/20	70.0
C-BESO	18/20	17/20	16/20	14/20	81.25
VQ-BeT	18/20	17/20	16/20	15/20	82.5
Mamba2Diff	20/20	18/20	18/20	17/20	91.25

Table 8

Params and Inference Times of Single Step for Mamba2Diff and baselines in the Real-World environments (both averaged with 20 runs each).

Method	Params(M)	Inference Times of Single Step (ms)	
		Scene 1	Scene2
CX-Diff	10.9	89.4	93.5
C-BESO	9.2	23.2	26.4
VQ-BeT	8.3	13.9	17.5
Mamba2Diff	7.7	12.6	14.1

Real-Time Efficiency Analysis: As is shown in Table 8, Mamba2Diff demonstrates superior real-time efficiency in real-world environments, achieving faster inference times with fewer parameters compared to the baselines. Its optimized structure has the potential to enable quick decision-making, making it highly suitable for long-horizon tasks that require real-time performance in dynamic, resource-constrained settings.

7. Conclusion

In this study, we propose Mamba2Diff, a novel diffusion framework that fuses multiscale dependencies for long-horizon robotic manipulation. By integrating the BDGM and CGRU modules, Mamba2Diff captures both long-term and short-term temporal features, enabling precise and consistent action generation in complex environments.

The framework is evaluated across three simulated environments-Block-Push, Multimodal Ant, and Relay Kitchen-as well as a custom real-world platform. Results show that Mamba2Diff achieves superior performance in terms of precision, efficiency, and temporal consistency. Specifically, success rate improvements over state-of-the-art methods are 2.7 %, 1.5 %, 2.2 %, and 8 % in the Block-Push, Ant, Kitchen, and real-world settings, respectively. In long-horizon tasks, Mamba2Diff

consistently outperforms diffusion-based and non-diffusion-based baselines across all stages of execution. Notably, in the final phases of task completion, it achieves success rate gains of 5 %, 4 %, and 12.5 % in the Kitchen, Ant, and real-world environments, respectively.

These findings highlight the effectiveness of Mamba2Diff's multi-scale modeling strategy and its potential for robust control in high-dimensional, dynamic tasks. The framework offers a solid foundation for real-world deployment in domains such as autonomous logistics and domestic assistance. Future work will explore its extension to broader robotic tasks.

CRedit authorship contribution statement

Guangpeng Zhao: Writing – review editing, Writing – original draft, Project administration, Methodology, Formal analysis, Data curation, Conceptualization; **Quan Liu:** Visualization, Validation, Supervision, Methodology; **Wupeng Deng:** Visualization, Validation, Resources; **Jiwei Hu:** Visualization, Validation, Supervision.

Data availability and access

Data will be made available on request. Data availability and access. The datasets generated during and/or analyzed during the current study are available from the corresponding author upon reasonable request.

Declaration of interests

Jiwei Hu reports financial support was provided by National Natural Science Foundation of China. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was fully supported by the [Natural Science Foundation of Hubei Province](#), China (Grant No. 2025AFA055).

References

- [1] S. Cheng, D. Xu, League: guided skill learning and abstraction for long-horizon manipulation, *IEEE Rob. Autom. Lett.* (2023).
- [2] H. Wang, H. Zhang, L. Li, Z. Kan, Y. Song, Task-driven reinforcement learning with action primitives for long-horizon manipulation skills, *IEEE Trans. Cybern.* (2023).
- [3] E. Triantafyllidis, F. Christianos, Z. Li, Intrinsic language-guided exploration for complex long-horizon robotic manipulation tasks, in: *2024 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2024, pp. 7493–7500.

- [4] C. Wang, L. Fan, J. Sun, R. Zhang, L. Fei-Fei, D. Xu, Y. Zhu, A. Anandkumar, Mimicplay: long-horizon imitation learning by watching human play, arXiv preprint arXiv:2302.12422 (2023).
- [5] J. Heo, Y.J. Lee, J. Kim, M.G. Kwak, Y.J. Park, S.B. Kim, Mixing corrupted preferences for robust and feedback-efficient preference-based reinforcement learning, *Knowl. Based Syst.* 309 (2025) 112824.
- [6] Z. Li, Y. Zhou, L. Wang, X. Zhang, A. Li, M. Zhu, Q. Wu, An end-to-end controller with image-based visual servoing of industrial manipulators with soft-actor-critic algorithm, *Knowl. Based Syst.* (2025) 112980.
- [7] S. Liu, Y. Liu, L. Hu, Z. Zhou, Y. Xie, Z. Zhao, W. Li, Z. Gan, Diffskill: improving reinforcement learning through diffusion-based skill denoiser for robotic manipulation, *Knowl. Based Syst.* 300 (2024) 112190.
- [8] Y. Ding, C. Florensa, P. Abbeel, M. Phielipp, Goal-conditioned imitation learning, *Adv. Neural. Inf. Process Syst.* 32 (2019).
- [9] H. Kim, Y. Ohmura, Y. Kuniyoshi, Goal-conditioned dual-action imitation learning for dexterous dual-arm robot manipulation, *IEEE Trans. Rob.* (2024).
- [10] J. Carvalho, A.T. Le, M. Baierl, D. Koert, J. Peters, Motion planning diffusion: learning and planning of robot motions with diffusion models, in: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2023, pp. 1916–1923.
- [11] J. Liu, M. Stamatiopoulou, D. Kanoulas, Dipper: diffusion-based 2d path planner applied on legged robots, in: 2024 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2024, pp. 9264–9270.
- [12] Z. Hou, T. Zhang, Y. Xiong, H. Pu, C. Zhao, R. Tong, Y. Qiao, J. Dai, Y. Chen, Diffus. Transf. Pol., arXiv preprint arXiv:2410.15959 (2024).
- [13] T. Dao, A. Gu, Transformers are SSMS: generalized models and efficient algorithms through structured state space duality, arXiv preprint arXiv:2405.21060 (2024).
- [14] Y. Zhao, B. Zhao, F. Zhang, C. He, W. Wu, L. Lai, SC-MAMBA2: Leveraging state-space models for efficient single-cell ultra-Long transcriptome modeling, *bioRxiv* (2024) 2024–09.
- [15] F. Zhang, N. Ji, F. Gao, B. Zhao, J. Wu, Y. Jiang, H. Du, Z. Ye, J. Zhu, W. Zhong, et al., DiM-gesture: co-speech gesture generation with adaptive layer normalization Mamba-2 framework, arXiv preprint arXiv:2408.00370 (2024).
- [16] J. Bai, Y. Fang, J. Wang, X. Zhang, A two-stage band-split Mamba-2 network for music separation, arXiv preprint arXiv:2409.06245 (2024).
- [17] Z. Zhang, A. Liu, I. Reid, R. Hartley, B. Zhuang, H. Tang, Motion mamba: efficient and long sequence motion generation, in: *European Conference on Computer Vision*, Springer, 2025, pp. 265–282.
- [18] J. Ho, S. Ermon, Generative adversarial imitation learning, *Adv. Neural. Inf. Process Syst.* 29 (2016).
- [19] M. Reuss, M. Li, X. Jia, R. Lioutikov, Goal-conditioned imitation learning using score-based diffusion policies, arXiv preprint arXiv:2304.02532 (2023).
- [20] A. Gu, T. Dao, Mamba: linear-time sequence modeling with selective state spaces, arXiv preprint arXiv:2312.00752 (2023).
- [21] A. Gu, K. Goel, C. Ré, Efficiently modeling long sequences with structured state spaces, arXiv preprint arXiv:2111.00396 (2021).
- [22] R. Xu, S. Yang, Y. Wang, B. Du, H. Chen, A survey on vision mamba: models, applications and challenges, arXiv preprint arXiv:2404.18861 (2024).
- [23] A. Gupta, V. Kumar, C. Lynch, S. Levine, K. Hausman, Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning, arXiv preprint arXiv:1910.11956 (2019).
- [24] C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine, P. Sermanet, Learning latent plans from play, in: *Conference on Robot Learning*, PMLR, 2020, pp. 1113–1132.
- [25] N.M. Shafiullah, Z. Cui, A.A. Altanzaya, L. Pinto, Behavior transformers: cloning k modes with one stone, *Adv. Neural. Inf. Process Syst.* 35 (2022) 22955–22968.
- [26] T. Pearce, T. Rashid, A. Kanervisto, D. Bignell, M. Sun, R. Georgescu, S.V. Macua, S.Z. Tan, I. Momennejad, K. Hofmann, et al., Imitating human behaviour with diffusion models, arXiv preprint arXiv:2301.10677 (2023).
- [27] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, S. Song, Diffusion policy: visuomotor policy learning via action diffusion, *Int. J. Rob. Res.* (2023) 02783649241273668.
- [28] P.M. Scheikl, N. Schreiber, C. Haas, N. Freymuth, G. Neumann, R. Lioutikov, F. Mathis-Ullrich, Movement primitive diffusion: learning gentle robotic manipulation of deformable objects, *IEEE Rob. Autom. Lett.* (2024).
- [29] J. Ho, A. Jain, P. Abbeel, Denoising diffusion probabilistic models, *Adv. Neural. Inf. Process Syst.* 33 (2020) 6840–6851.
- [30] V. Saxena, Y. Koga, D. Xu, Constrained-context conditional diffusion models for imitation learning, arXiv preprint arXiv:2311.01419 (2023).
- [31] F. Nuti, T. Franzmeyer, J.F. Henriques, Extracting reward functions from diffusion models, *Adv. Neural. Inf. Process Syst.* 36 (2024).
- [32] P. Wang, Z. Guo, A.L. Sait, M.H. Pham, Robot Shape and Location Retention in Video Generation Using Diffusion Models, arXiv preprint arXiv:2407.02873 (2024).
- [33] Z. Xian, N. Gkanatsios, T. Gervet, T.-W. Ke, K. Fragkiadaki, Chaineddiffuser: unifying trajectory diffusion and keypose prediction for robotic manipulation, in: 7th Annual Conference on Robot Learning, 2023.
- [34] T. Karras, M. Aittala, T. Aila, S. Laine, Elucidating the design space of diffusion-based generative models, *Adv. Neural. Inf. Process Syst.* 35 (2022) 26565–26577.
- [35] Y. Nung, S. Ermon, Generative modeling by estimating gradients of the data distribution, *Adv. Neural. Inf. Process Syst.* 32 (2019).
- [36] T.R. Mim, M. Amatullah, S. Afreen, M.A. Yousuf, S. Uddin, S.A. Alyami, K.F. Hasan, M.A. Moni, GRU-INC: an inception-attention based approach using GRU for human activity recognition, *Expert Syst. Appl.* 216 (2023) 119419.
- [37] C. Tang, L. Xu, B. Yang, Y. Tang, D. Zhao, GRU-Based interpretable multivariate time series anomaly detection in industrial control system, *Comp. Secur.* 127 (2023) 103094.
- [38] Z. Zhao, S. Yun, L. Jia, J. Guo, Y. Meng, N. He, X. Li, J. Shi, L. Yang, Hybrid VMD-CNN-GRU-based model for short-term forecasting of wind power considering spatio-temporal features, *Eng. Appl. Artif. Intell.* 121 (2023) 105982.
- [39] W. Huang, J. Hu, ML-Mamba: efficient multi-modal large language model utilizing Mamba-2, arXiv preprint arXiv:2407.19832 (2024).
- [40] X. Fengchang, R. Alfred, R.H. Pailus, J.V.L. Chew, L. Ge, L. Guozhang, D. Shifeng, W. Xinliang, Research on DETR novel small target detection algorithm based on swin transformer, *IEEE Access* (2024).
- [41] J. Song, C. Meng, S. Ermon, Denoising diffusion implicit models, arXiv preprint arXiv:2010.02502 (2020).
- [42] O. Mees, L. Hermann, W. Burgard, What matters in language conditioned robotic imitation learning over unstructured data, *IEEE Rob. Autom. Lett.* 7 (4) (2022) 11205–11212.
- [43] P. Florence, C. Lynch, A. Zeng, O.A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, J. Tompson, Implicit behavioral cloning, in: *Conference on Robot Learning*, PMLR, 2022, pp. 158–168.
- [44] S. Lee, Y. Wang, H. Etukuru, H.J. Kim, N.M.M. Shafiullah, L. Pinto, Behavior generation with latent actions, arXiv preprint arXiv:2403.03181 (2024).
- [45] Z.J. Cui, Y. Wang, N.M.M. Shafiullah, L. Pinto, From play to policy: Conditional behavior generation from uncensored robot data, arXiv preprint arXiv:2210.10047 (2022).
- [46] G. Brockman, OpenAI Gym, arXiv preprint arXiv:1606.01540 (2016).
- [47] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, J. Zhu, Dpm-solver: a fast ode solver for diffusion probabilistic model sampling in around 10 steps, *Adv. Neural. Inf. Process Syst.* 35 (2022) 5775–5787.
- [48] W. Zhao, L. Bai, Y. Rao, J. Zhou, J. Lu, Unipc: a unified predictor-corrector framework for fast sampling of diffusion models, *Adv. Neural. Inf. Process Syst.* 36 (2024).
- [49] K. Zheng, C. Lu, J. Chen, J. Zhu, Dpm-solver-v3: improved diffusion ode solver with empirical model statistics, *Adv. Neural. Inf. Process Syst.* 36 (2023) 55502–55542.
- [50] P. Wu, Y. Shentu, Z. Yi, X. Lin, P. Abbeel, Gello: A general, low-cost, and intuitive teleoperation framework for robot manipulators, arXiv preprint arXiv:2309.13037 (2023).
- [51] M. Reuss, Ö.E. Yağmurlu, F. Wenzel, R. Lioutikov, Multimodal diffusion transformer: learning versatile behavior from multimodal goals, in: *First Workshop on Vision-Language Models for Navigation and Manipulation at ICRA 2024*, 2024.